UNIT- V

Real Time Operating Systems: Brief History of OS - Defining RTOS - The Scheduler - Objects – Services - Characteristics of RTOS - Defining a Task - Tasks States and Scheduling - Task Operations – Structure – Synchronization - Communication and Concurrency. Defining Semaphores - Operations and Use - Defining Message Queue - States – Content – Storage - Operations and Use

BRIEF HISTORY OF OS:

- A real-time operating system (RTOS) is key to many embedded systems today and, provides a software platform upon which to build applications.
- Not all embedded systems, however, are designed with an RTOS.
- Some embedded systems with relatively simple hardware or a small amount of software application code might not require an RTOS.
- Many embedded systems, however, with moderate-to-large software applications require some form of scheduling, and these systems require an RTOS.
- In the early days of computing, developers created software applications that included low-level machine code to initialize and interact with the system's hardware directly.
- This tight integration between the software and hardware resulted in non-portable applications.
- A small change in the hardware might result in rewriting much of the application itself. Obviously, these systems were difficult and costly to maintain.
- Over the years, many versions of operating systems evolved. These ranged from general-purpose operating systems (GPOS), such as UNIX and Microsoft Windows, to smaller and more compact real-time operating systems, such as VxWorks

- In the 60s and 70s, when mid-sized and mainframe computing was in its prime, UNIX was developed to facilitate multi-user access to expensive, limited-availability computing systems.

- UNIX allowed many users performing a variety of tasks to share these large and costly computers. multi-user access was very efficient: one user could print files, for example, while another wrote programs.
- Eventually, UNIX was ported to all types of machines, from microcomputers to supercomputers.
- In the 80s, Microsoft introduced the Windows operating system, which emphasized the personal computing environment.
- Targeted for residential and business users interacting with PCs through a graphical user interface, the Microsoft Windows operating system helped drive the personal-computing era.
- Some core functional similarities between a typical RTOS and GPOS include:
  - some level of multitasking,
  - software and hardware resource management,
  - provision of underlying OS services to applications, and
  - abstracting the hardware from the software application.

- On the other hand, some key functional differences that set RTOSes apart from GPOSes include:
  - better reliability in embedded application contexts,
  - the ability to scale up or down to meet application needs,
  - faster performance,
  - reduced memory requirements,
  - scheduling policies tailored for real-time embedded systems,
  - support for diskless embedded systems by allowing executables to boot and run from ROM or RAM, and
  - better portability to different hardware platforms.

- GPOSes typically require a lot more memory, however, and are not well suited to real-time embedded devices with limited memory and high performance requirements.
- RTOSes, on the other hand, can meet these requirements. They are reliable, compact, and scalable, and they perform well in real-time embedded systems.
- In addition, RTOSes can be easily tailored to use only those components required for a particular application.

DEFINING RTOS:

- A real-time operating system (RTOS) is a program that schedules execution in a timely manner, manages system resources, and provides a consistent foundation for developing application code.
- Application code designed on an RTOS can be quite diverse, ranging from a simple application for a digital stopwatch to a much more complex application for aircraft navigation. Good RTOSes, therefore, are scalable in order to meet different sets of requirements for different applications.
- For example, in some applications, an RTOS comprises only a kernel, which is the core supervisory software that provides minimal logic, scheduling, and resource-management algorithms.
- Every RTOS has a kernel. On the other hand, an RTOS can be a combination of various modules, including the kernel, a file system, networking protocol stacks, and other components required for a particular application, as illustrated at a high level in figure.

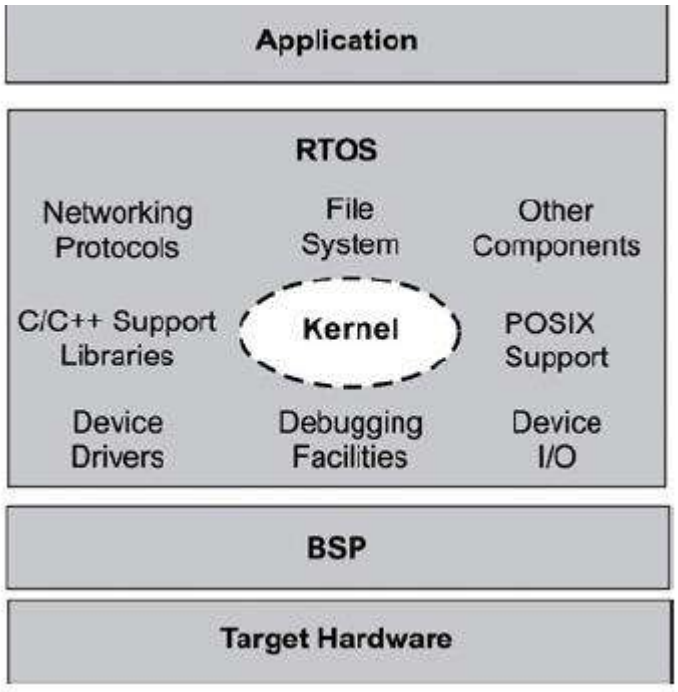


Figure: High-level view of an RTOS, its kernel, and other components found in embedded systems.

- Although many RTOSes can scale up or down to meet application requirements, this book focuses on the common element at the heart of all RTOSes-the kernel. Most RTOS kernels contain the following components:
- **Scheduler**-is contained within each kernel and follows a set of algorithms that determines which task executes when. Some common examples of scheduling algorithms include round-robin and preemptive scheduling.
- Objects -are special kernel constructs that help developers create applications for real-time embedded systems. Common kernel objects include tasks, semaphores, and message queues.

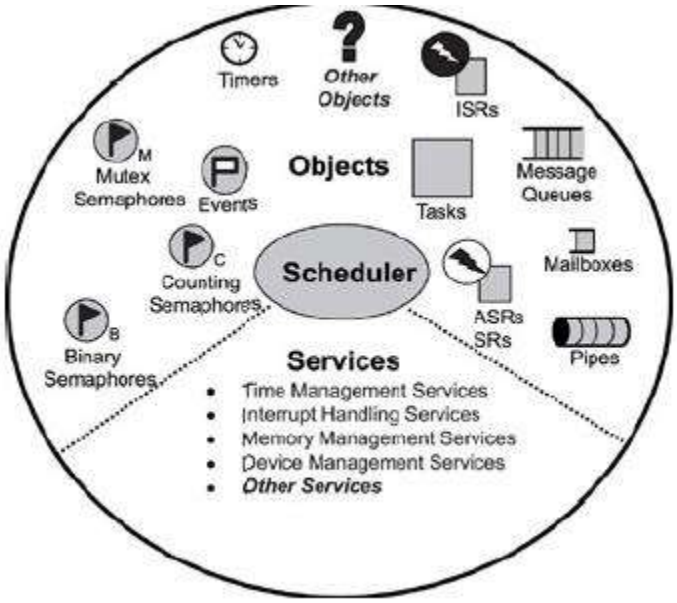


Figure : Common components in an RTOS kernel that including objects, the scheduler, and some services.

- This diagram is highly simplified; remember that not all RTOS kernels conform to this exact set of objects, scheduling algorithms, and services.

THE SCHEDULER:

- The scheduler is at the heart of every kernel. A scheduler provides the algorithms needed to determine which task executes when. To understand how scheduling works, this section describes the following topics:

  - schedulable entities,
  - multitasking,
  - context switching,
  - dispatcher, and

- scheduling algorithms.

SCHEDULABLE ENTITIES:

- A schedulable entity is a kernel object that can compete for execution time on a system, based on a predefined scheduling algorithm. Tasks and processes are all examples of schedulable entities found in most kernels.
- A task is an independent thread of execution that contains a sequence of independently schedulable instructions.
- Some kernels provide another type of a schedulable object called a process. Processes are similar to tasks in that they can independently compete for CPU execution time.
- Processes differ from tasks in that they provide better memory protection features, at the expense of performance and memory overhead. Despite these differences, for the sake of simplicity, this book uses task to mean either a task or a process.

MULTITASKING:

- Multitasking is the ability of the operating system to handle multiple activities within set deadlines. A real-time kernel might have multiple tasks that it has to schedule to run. One such multitasking scenario is illustrated in figure.

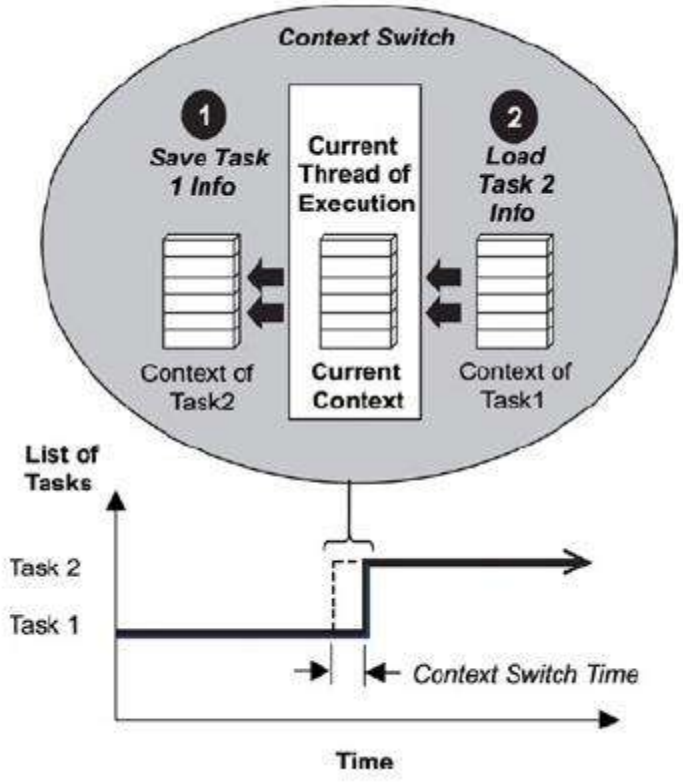


Figure : Multitasking using a context switch.

- In this scenario, the kernel multitasks in such a way that many threads of execution appear to be running concurrently; however, the kernel is actually interleaving executions sequentially, based on a preset scheduling algorithm.
- An important point to note here is that the tasks follow the kernel s scheduling algorithm, while interrupt service routines (ISR) are triggered to run because of hardware interrupts and their established priorities.

THE CONTEXT SWITCH:

- Each task has its own context, which is the state of the CPU registers required each time it is scheduled to run.
- A context switch occurs when the scheduler switches from one task to another. To better understand what happens during a context switch, let s examine further what a typical kernel does in this scenario.
- As shown in the above figure, when the kernel s scheduler determines that it needs to stop running task 1 and start running task 2, it takes the following steps:

    - The kernel saves task 1 s context information in its TCB.
    - It loads task 2 s context information from its TCB, which becomes the current thread of execution.

    3.The context of task 1 is frozen while task 2 executes, but if the scheduler needs to run task 1 again , task 1 continues from where it left off just before the context switch.

- The time it takes for the scheduler to switch from one task to another is the context switch time.
- It is relatively insignificant compared to most operations that a task performs.
- If an application s design includes frequent context switching, however, the application can incur unnecessary performance overhead. Therefore, design applications in a way that does not involve excess context switching.

THE DISPATCHER:

- The dispatcher is the part of the scheduler that performs context switching and changes the flow of execution. At any time an RTOS is running, the flow of execution, also known as flow of control, is passing through one of three areas: through an application task, through an ISR, or through the kernel.

- Depending on how the kernel is first entered, dispatching can happen differently. When a task makes system calls, the dispatcher is used to exit the kernel after every system call completes.

SCHEDULING ALGORITHMS:

- The scheduler determines which task runs by following a scheduling algorithm (also known as scheduling policy). Most kernels today support two common scheduling algorithms:
- ➢ preemptive priority-based scheduling, and round-robin scheduling.

PREEMPTIVE PRIORITY-BASED SCHEDULING:

- Most real-time kernels use preemptive priority-based scheduling by default.
- As shown in figure with this type of scheduling, the task that gets to run at any point is the task with the highest priority among all other tasks ready to run in the system.

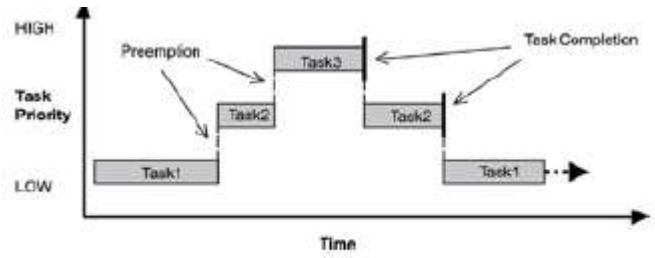


Figure: Preemptive priority-based scheduling.

- Real-time kernels generally support 256 priority levels, in which 0 is the highest and 255 the lowest. Some kernels appoint the priorities in reverse order; where 255 is the highest and 0 the lowest.
- Regardless, the concepts are basically the same. With a preemptive priority-based scheduler, each task has a priority, and the highest-priority task runs first.
- If a task with a priority higher than the current task becomes ready to run, the kernel immediately saves the current task s context in its TCB and switches to the higher-priority task.
- As shown in figure task 1 is preempted by higher-priority task 2, which is then preempted by task 3.
- When task 3 completes, task 2 resumes; likewise, when task 2 completes, task 1 resumes.

ROUND-ROBIN SCHEDULING:

- Round-robin scheduling provides each task an equal share of the CPU execution time. Pure round-robin scheduling cannot satisfy real-time system requirements because in real-time systems, tasks perform work of varying degrees of importance.
- Instead, preemptive, priority-based scheduling can be augmented with round-robin scheduling which uses time slicing to achieve equal allocation of the CPU for tasks of the same priority as shown in figure.

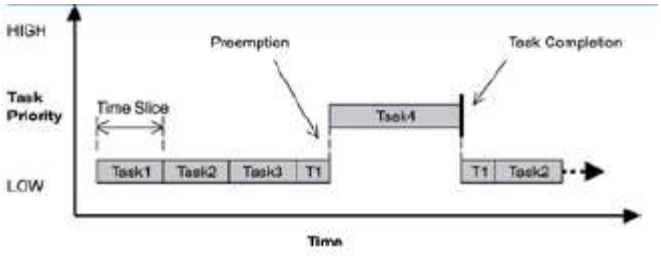


Figure : Round-robin and preemptive scheduling.

- With time slicing, each task executes for a defined interval, or time slice, in an ongoing cycle, which is the round robin. A run-time counter tracks the time slice for each task, incrementing on every clock tick.
- When one task s time slice completes, the counter is cleared, and the task is placed at the end of the cycle. Newly added tasks of the same priority are placed at the end of the cycle, with their run-time counters initialized to 0.
- If a task in a round-robin cycle is preempted by a higher-priority task, its run-time count is saved and then restored when the interrupted task is again eligible for execution.
- This idea is illustrated in figure, in which task 1 is preempted by a higher-priority task 4 but resumes where it left off when task 4 completes.

OBJECTS:

- Kernel objects are special constructs that are the building blocks for application development for real-time embedded systems.
- The most common RTOS kernel objects are

➢ Tasks are concurrent and independent threads of execution that can compete for CPU execution time.

➢ Semaphores are token-like objects that can be incremented or decremented by tasks for synchronization or mutual exclusion.

➢ Message Queues are buffer-like data structures that can be used for synchronization, mutual exclusion, and data exchange by passing messages between tasks.

- Developers creating real-time embedded applications can combine these basic kernel objects (as well as others not mentioned here) to solve common real-time design problems, such as concurrency, activity synchronization, and data communication.

SERVICES:

- Along with objects, most kernels provide services that help developers create applications for real-time embedded systems.
- These services comprise sets of API calls that can be used to perform operations on kernel objects or can be used in general to facilitate timer management, interrupt handling, device I/O, and memory management.
- Again, other services might be provided; these services are those most commonly found in RTOS kernels.

CHARACTERISTICS OF AN RTOS:

- An application's requirements define the requirements of its underlying RTOS. Some of the more common attributes are:
  - reliability,
  - predictability,
  - performance,
  - compactness, and
  - scalability.

- These attributes are discussed next; however, the RTOS attribute an application needs depends on the type of application being built.

RELIABILITY:

- Embedded systems must be reliable. Depending on the application, the system might need to operate for long periods without human intervention.
- Different degrees of reliability may be required. For example, a digital solar-powered calculator might reset itself if it does not get enough light, yet the calculator might still be considered acceptable.
- On the other hand, a telecom switch cannot reset during operation without incurring high associated costs for down time. The RTOSes in these applications require different degrees of reliability.
- Although different degrees of reliability might be acceptable, in general, a reliable system is one that is available (continues to provide service) and does not fail.

PREDICTABILITY:

- Because many embedded systems are also real-time systems, meeting time requirements is key to ensuring proper operation. The RTOS used in this case needs to be predictable to a certain degree.
- The term deterministic describes RTOSes with predictable behavior, in which the completion of operating system calls occurs within known timeframes.
- Developers can write simple benchmark programs to validate the determinism of an RTOS. The result is based on timed responses to specific RTOS calls.
- In a good deterministic RTOS, the variance of the response times for each type of system call is very small.

PERFORMANCE:

- This requirement dictates that an embedded system must perform fast enough to fulfill its timing requirements. Typically, the more deadlines to be met-and the shorter the time between them-the faster the system's CPU must be.
- Although underlying hardware can dictate a system's processing power, its software can also contribute to system performance. Typically, the processor's performance is expressed in million instructions per second (MIPS).
- Throughput also measures the overall performance of a system, with hardware and software combined. One definition of throughput is the rate at which a system can generate output based on the inputs coming in.
- Throughput also means the amount of data transferred divided by the time taken to transfer it. Data transfer throughput is typically measured in multiples of bits per second (bps).
- Sometimes developers measure RTOS performance on a call-by-call basis. Benchmarks are written by producing timestamps when a system call starts and when it completes.
- Although this step can be helpful in the analysis stages of design, true performance testing is achieved only when the system performance is measured as a whole.

COMPACTNESS:

- Application design constraints and cost constraints help determine how compact an embedded system can be. For example, a cell phone clearly must be small, portable, and low cost. These design requirements limit system memory, which in turn limits the size of the application and operating system.
- In such embedded systems, where hardware real estate is limited due to size and costs, the RTOS clearly must be small and efficient. In these cases, the RTOS memory footprint can be an important factor.

- To meet total system requirements, designers must understand both the static and dynamic memory consumption of the RTOS and the application that will run on it.

SCALABILITY:

- Because RTOSes can be used in a wide variety of embedded systems, they must be able to scale up or down to meet application-specific requirements.
- Depending on how much functionality is required, an RTOS should be capable of adding or deleting modular components, including file systems and protocol stacks.
- If an RTOS does not scale up well, development teams might have to buy or build the missing pieces. Suppose that a development team wants to use an RTOS for the design of a cellular phone project and a base station project.
- If an RTOS scales well, the same RTOS can be used in both projects, instead of two different RTOSes, which saves considerable time and money.

DEFINING A TASK:

- A *task* is an independent thread of execution that can compete with other concurrent tasks for processor execution time. As mentioned earlier, developers decompose applications into multiple concurrent tasks to optimize the handling of inputs and outputs within set time constraints.
- A task is *schedulable* , the task is able to compete for execution time on a system, based on a predefined scheduling algorithm. A task is defined by its distinct set of parameters and supporting data structures.
- Specifically, upon creation, each task has an associated name, a unique ID, a priority (if part of a preemptive scheduling plan), a task control block (TCB), a stack, and a task routine, as shown in figure. Together, these components make up what is known as the *task object.*

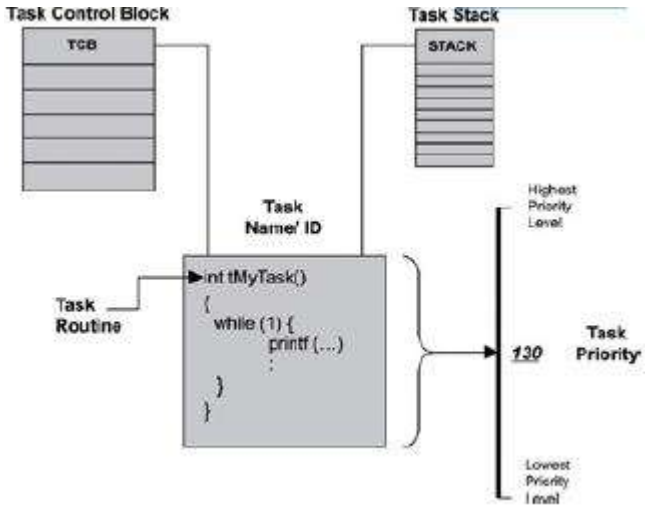


Figure : A task, its associated parameters, and supporting data structures.

- When the kernel first starts, it creates its own set of *system tasks* and allocates the appropriate priority for each from a set of *reserved priority levels.*
- The reserved priority levels refer to the priorities used internally by the RTOS for its system tasks. An application should avoid using these priority levels for its tasks because running application tasks at such level may affect the overall system performance or behavior.
- For most RTOSes, these reserved priorities are not enforced. The kernel needs its system tasks and their reserved priority levels to operate. These priorities should not be modified. Examples of system tasks include:

  - **initialization or startup task** initializes the system and creates and starts system tasks,
  - **idle task** uses up processor idle cycles when no other activity is present,
  - **logging task** logs system messages,
  - **exception-handling task** handles exceptions, and
  - **debug agent task** allows debugging with a host debugger.

- The idle task, which is created at kernel startup, is one system task that bears mention and should not be ignored. The idle task is set to the lowest priority, typically executes in an endless loop, and runs when either no other task can run or when no other tasks exist, for the sole purpose of using idle processor cycles.
- The idle task is necessary because the processor executes the instruction to which the program counter register points while it is running. Unless the processor can be suspended, the program counter must still point to valid instructions even when no tasks exist in the system or when no tasks can run.
- Therefore, the idle task ensures the processor program counter is always valid when no other tasks are running. In some cases, however, the kernel might allow a user-configured routine to run instead of the idle task in order to implement special requirements for a particular application. One example of a special requirement is power conservation.
- When no other tasks can run, the kernel can switch control to the user-supplied routine instead of to the idle task. In this case, the user-supplied routine acts like the idle task but instead initiates power conservation code, such as system suspension, after a period of idle time.
- After the kernel has initialized and created all of the required tasks, the kernel jumps to a predefined entry point (such as a predefined function) that serves, in effect, as the beginning of

the application. From the entry point, the developer can initialize and create other application tasks, as well as other kernel objects, which the application design might require.

- As the developer creates new tasks, the developer must assign each a task name, priority, stack size, and a task routine. The kernel does the rest by assigning each task a unique ID and creating an associated TCB and stack space in memory for it.

TASK STATES AND SCHEDULING:

- Whether it's a system task or an application task, at any time each task exists in one of a small number of states, including ready, running, or blocked. As the real-time embedded system runs, each task moves from one state toanother, according to the logic of a simple finite state machine (FSM). Figure illustrates a typical FSM for task execution states, with brief descriptions of state transitions.

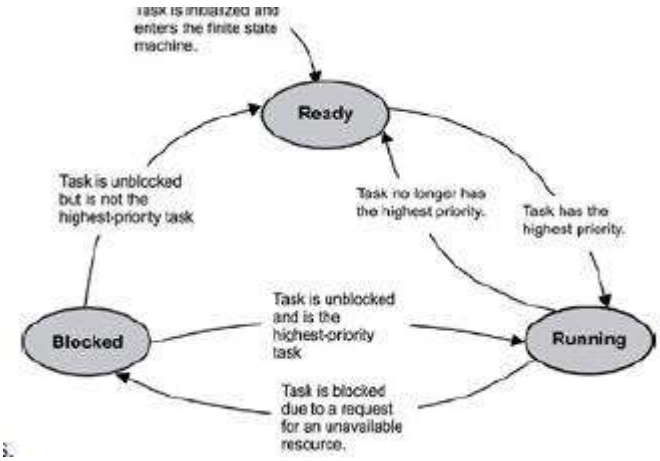


Figure: A typical finite state machine for task execution states.

- Although kernels can define task-state groupings differently, generally three main states are used in most typical preemptive-scheduling kernels, including:

➢ **ready state**-the task is ready to run but cannot because a higher priority task is executing.

- **blocked state**-the task has requested a resource that is not available, has requested to wait until some event occurs, or has delayed itself for some duration.

- **running state**-the task is the highest priority task and is running.

- In some cases, the kernel changes the states of some tasks, but no context switching occurs because the state of the highest priority task is unaffected.
- In other cases, however, these state changes result in a context switch because the former highest priority task either gets blocked or is no longer the highest priority task.
- When this process happens, the former running task is put into the blocked or ready state, and the new highest priority task starts to execute.

READY STATE:

- When a task is first created and made ready to run, the kernel puts it into the ready state. In this state, the task actively competes with all other ready tasks for the processor's execution time. As figure shows, tasks in the ready state cannot move directly to the blocked state.
- A task first needs to run so it can make a *blocking call* , which is a call to a function that cannot immediately run to completion, thus putting the task in the blocked state.
- Ready tasks, therefore, can only move to the running state. Because many tasks might be in the ready state, the kernel's scheduler uses the priority of each task to determine which task to move to the running state.
- Figure illustrates, in a five-step scenario, how a kernel scheduler might use a task-ready list to move tasks from the ready state to the running state.
- This example assumes a single-processor system and a priority-based preemptive scheduling algorithm in which 255 is the lowest priority and 0 is the highes

Figure : Five steps showing the way a task-ready list works.

- In this example, tasks 1, 2, 3, 4, and 5 are ready to run, and the kernel queues them by priority in a task-ready list.
- Task 1 is the highest priority task (70); tasks 2, 3, and 4 are at the next-highest priority level (80); and task 5 is the lowest priority (90). The following steps explains how a kernel might use the task-ready list to move tasks to and from the ready state:
  - Tasks 1, 2, 3, 4, and 5 are ready to run and are waiting in the task-ready list.
  - Because task 1 has the highest priority (70), it is the first task ready to run. If nothing higher is running, the kernel removes task 1 from the ready list and moves it to the running state. During execution, task 1 makes a blocking call. As a result, the kernel moves task 1 to the blocked state; takes task 2, which is first in the list of the next-highest priority tasks (80), off the ready list; and moves task 2
    to the running state.
  - Next, task 2 makes a blocking call. The kernel moves task 2 to the blocked state; takes task 3, which is next in line of the priority 80 tasks, off the ready list; and moves task 3 to the running state.
  - As task 3 runs, frees the resource that task 2 requested. The kernel returns task 2 to the ready state and inserts it at the end of the list of tasks ready to run at priority level 80. Task 3 continues as the currently running task.

RUNNING STATE:

- On a single-processor system, only one task can run at a time. In this case, when a task is moved to the running state, the processor loads its registers with this task's context.
- The processor can then execute the task's instructions and manipulate the associated stack. Unlike a ready task, a running task can move to the blocked state in any of the following ways:

➢ by making a call that requests an unavailable resource,

➢ by making a call that requests to wait for an event to occur, and

➢ by making a call to delay the task for some duration.

➢ In each of these cases, the task is moved from the running state to the blocked state, as described next.

BLOCKED STATE:

- The possibility of blocked states is extremely important in real-time systems because without blocked states, lower priority tasks could not run. If higher priority tasks are not designed to block, CPU starvation can result.
- *CPU starvation* occurs when higher priority tasks use all of the CPU execution time and lower priority tasks do not get to run.
- A task can only move to the blocked state by making a blocking call, requesting that some blocking condition be met. A blocked task remains blocked until the blocking condition is met. (It probably ought to be called the *un*blocking condition, but blocking is the terminology in common use among real-time programmers.) Examples of how blocking conditions are met include the following:

➢ a semaphore token (described later) for which a task is waiting is released,

➢ a message, on which the task is waiting, arrives in a message queue, or

➢ a time delay imposed on the task expires.

TASK OPERATIONS:

- In addition to providing a task object, kernels also provide *task-management services* . Task-management services include the actions that a kernel performs behind the scenes to support tasks, for example, creating and maintaining the TCB and task stacks.

- A kernel, however, also provides an API that allows developers to manipulate tasks. Some of the more common operations that developers can perform with a task object from within the application include:

➢ creating and deleting tasks,

➢ controlling task scheduling, and

➢ obtaining task information.

THE TASK CREATION AND DELETION:

The most fundamental operations that developers must learn are creating and deleting tasks, as shown in Table

Operation	Description
Create	Creates a task
Delete	Deletes a task

.

- Developers typically create a task using one or two operations, depending on the kernel s API. Some kernels allow developers first to create a task and then start it. In this case, the task is first created and put into a suspended state; then, the task is moved to the ready state when it is started (made ready to run).
- Creating tasks in this manner might be useful for debugging or when special initialization needs to occur between the times that a task is created and started. However, in most cases, it is sufficient to create and start a task using one kernel call.
- Starting a task does not make it run immediately; it puts the task on the task-ready list.
- Many kernels also provide *user-configurable hooks* , which are mechanisms that execute programmer-supplied functions, at the time of specific kernel events. The programmer *registers* the function with the kernel by passing a function pointer to a kernel-provided API . The kernel executes this function when the event of interest occurs. Such events can include:

➢ when a task is first created,

➢ when a task is suspended for any reason and a context switch occurs, and

➢ when a task is deleted.

- However, when tasks execute, they can acquire memory or access resources using other kernel objects. If the task is deleted incorrectly, the task might not get to release these resources. For example, assume that a task acquires a semaphore token to get exclusive access to a shared data structure.
- While the task is operating on this data structure, the task gets deleted. If not handled appropriately, this abrupt deletion of the operating task can result in:

- a corrupt data structure, due to an incomplete write operation,
- an unreleased semaphore, which will not be available for other tasks that might need to acquire it, and
- an inaccessible data structure, due to the unreleased semaphore.

- As a result, premature deletion of a task can result in memory or resource leaks. A m*emory leak* occurs when memory is acquired but not released, which causes the system to run out of memory eventually.
- A *resource leak* occurs when a resource is acquired but never released, which results in a memory leak because each resource takes up space in memory. Many kernels provide *task-deletion locks*, a pair of calls that protect a task from being prematurely deleted during a critical section of code.

TASK SCHEDULING:

- From the time a task is created to the time it is deleted, the task can move through various states resulting from program execution and kernel scheduling.
- Although much of this state changing is automatic, many kernels provide a set of API calls that allow developers to control when a task moves to a different state, as shown in Table . This

capability is called *manual scheduling*

Operation	Description
Suspend	Suspends a task
Resume	Resumes a task
Delay	Delays a task
Restart	Restarts a task
Get Priority	Gets the current task s priority
Set Priority	Dynamically sets a task s priority
Preemption lock	Locks out higher priority tasks from preempting the current task
Preemption unlock	Unlocks a preemption lock

- A developer might want to delay (block) a task, for example, to allow manual scheduling or to wait for an external condition that does not have an associated interrupt. Delaying a task causes it to relinquish the CPU and allow another task to execute.
- After the delay expires, the task is returned to the task-ready list after all other ready task sat its priority level. A delayed task waiting for an external condition can wake up after a set time to check whether a specified condition or event has occurred, which is called polling.
- Getting and setting a task s priority during execution lets developers control task scheduling manually. This process is helpful during a priority inversion, in which a lower priority task has a shared resource that a higher priority task requires and is preempted by an unrelated medium-priority task.
- Finally, the kernel might support preemption locks, a pair of calls used to disable and enable preemption in applications. This feature can be useful if a task is executing in a critical section of code: one in which the task must not be preempted by other tasks.

OBTAINING TASK INFORMATION:

- Kernels provide routines that allow developers to access task information within their applications, as shown in Table. This information is useful for debugging and monitoring.

.

Operation	Description
Get ID	Get the current task s ID
Get TCB	Get the current task s TCB

Table : Task-information operations.

- One use is to obtain a particular task s ID, which is used to get more information about the task by getting its TCB.
- Obtaining a TCB, however, only takes a snapshot of the task context. If a task is not dormant (e.g., suspended), its context might be dynamic, and the snapshot information might change by the time it is used.
- Hence, use this functionality wisely, so that decisions are to made in the application based on querying a constantly changing task context.

STRUCTURE:

- When writing code for tasks, tasks are structured in one of two ways:
  - run to completion, or
  - endless loop.

- Both task structures are relatively simple. Run-to-completion tasks are most useful for initialization and startup. They typically run once, when the system first powers on. Endless-loop tasks do the majority of the work in the application by handling inputs and outputs. Typically, they run many times while the system is powered on.

RUN-TO-COMPLETION TASKS:

- An example of a run-to-completion task is the application-level initialization task, shown in Listing . The initialization task initializes the application and creates additional services, tasks, and needed kernel objects.

Listing 5.1: Pseudo code for a run-to-completion task.

```
RunToCompletionTask ()
{
    Initialize application
    Create  endless loop tasks'
    Create kernel objects
    Delete or suspend this task
}
```

- The application initialization task typically has a higher priority than the application tasks it creates so that its initialization work is not preempted. In the simplest case, the other tasks are one or more lower priority endless-loop tasks.
- The application initialization task is written so that it suspends or deletes itself after it completes its work so the newly created tasks can run.

ENDLESS-LOOP TASKS:

- As with the structure of the application initialization task, the structure of an endless loop task can also contain initialization code. The endless loop's initialization code, however, only needs to be executed when the task first runs, after which the task executes in an endless loop, as shown in Listing .
- The critical part of the design of an endless-loop task is the one or more blocking calls within the body of the loop.
- These blocking calls can result in the blocking of this endless-loop task, allowing lower priority tasks to run. Listing : Pseudo code for an endless-loop task.

```
EndlessLoopTask ()
{
    Initialization code
    Loop Forever

    {
        Body of loop
        Make one or more blocking calls
    }
}
```

SYNCHRONIZATION:

- Synchronization is classified into two categories: *resource synchronization* and *activity synchronization* . Resource synchronization determines whether access to a shared resource is safe, and, if not, when it will be safe.
- Activity synchronization determines whether the execution of a multithreaded program has reached a certain state and, if it hasn't, how to wait for and be notified when this state is reached.

RESOURCE SYNCHRONIZATION:

- Access by multiple tasks must be synchronized to maintain the integrity of a shared resource. This process is called *resource synchronization* , a term closely associated with critical sections and mutual exclusions.
- *Mutual exclusion* is a provision by which only one task at a time can access a shared resource. A *critical section* is the section of code from which the shared resource is accessed.

- As an example, consider two tasks trying to access shared memory. One task (the sensor task) periodically receives data from a sensor and writes the data to shared memory. Meanwhile, a second task (the display task) periodically reads from shared memory and sends the data to a display. The common design pattern of using shared memory is illustrated in Figure.

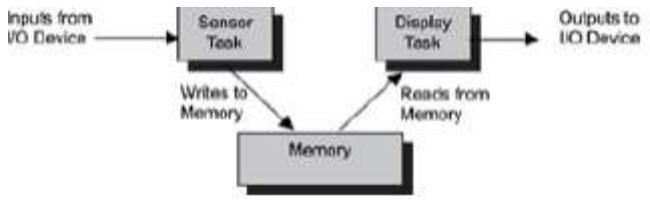


Figure : Multiple tasks accessing shared memory.

- The section of code in the sensor task that writes input data to the shared memory is a critical section of the sensor task. The section of code in the display task that reads data from the shared memory is a critical section of the display task.
- These two critical sections are called competing critical sections because they access the same shared resource.
- A mutual exclusion algorithm ensures that one task's execution of a critical section is not interrupted by the competing critical sections of other concurrently executing tasks.

- One way to synchronize access to shared resources is to use a client-server model, in which a central entity called a resource server is responsible for synchronization. Access requests are

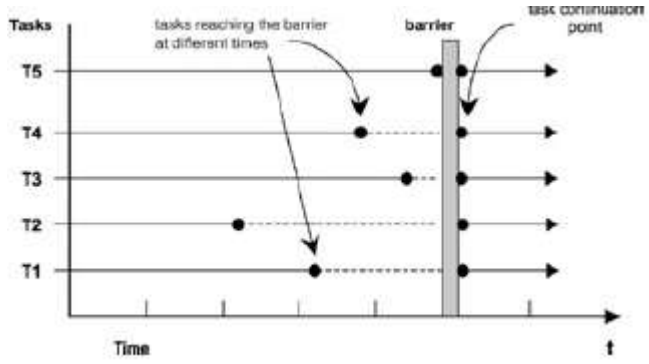
made to the resource server, which must grant permission to the requestor before the requestor can access the shared resource.

- The resource server determines the eligibility of the requestor based on pre-assigned rules or run-time heuristics.

ACTIVITY SYNCHRONIZATION:

- In general, a task must synchronize its activity with other tasks to execute a multithreaded program properly. Activity synchronization is also called condition synchronization or sequence control. Activity synchronization ensures that the correct execution order among cooperating tasks is used. Activity synchronization can be either synchronous or asynchronous.
- One representative of activity synchronization methods is barrier synchronization. The point at which the partial results are collected and the duration of the final computation is a barrier. One task can finish its partial computation before other tasks complete theirs, but this task must wait for all other tasks to complete their computations before the task can continue.
- Barrier synchronization comprises three actions:

  - a task posts its arrival at the barrier,
  - the task waits for other participating tasks to reach the barrier, and
  - the task receives notification to proceed beyond the barrier.



- As shown in Figure, a group of five tasks participates in barrier synchronization. Tasks in the group complete their partial execution and reach the barrier at various times; however, each task in the group must wait at the barrier until all other tasks have reached the barrier.
- The last task to reach the barrier (in this example, task T5) broadcasts a notification to the other tasks. All tasks cross the barrier at the same time (conceptually in a uniprocessor environment due to task scheduling.
- Another representative of activity synchronization mechanisms is *rendezvous* synchronization, which, as its name implies, is an execution point where two tasks meet. The main difference

between the barrier and the rendezvous is that the barrier allows activity synchronization among two or more tasks, while rendezvous synchronization is between two tasks.

- In rendezvous synchronization, a synchronization and communication point called an *entry* is constructed as a function call. One task defines its entry and makes it public. Any task with knowledge of this entry can call it as an ordinary function call. The task that defines the entry accepts the call, executes it, and returns the results to the caller.
- The issuer of the entry call establishes a rendezvous with the task that defined the entry.
- A derivative form of rendezvous synchronization, called simple rendezvous in this book, uses kernel primitives, such as semaphores or message queues, instead of the entry call to achieve synchronization. Two tasks can implement a simple rendezvous without data passing by using two binary semaphores, as shown in Figure.

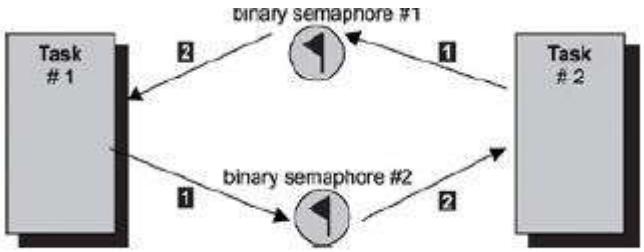


Figure : Simple rendezvous without data passing.

IMPLEMENTING BARRIERS:

- Barrier synchronization is used for activity synchronization. listing shows how to implement a barrier-synchronization mechanism using a mutex and a condition variable.

```
typedef struct {
mutex_t br_lock; /* guarding mutex */
cond_t br_cond; /* condition variable */
int br_count; /* num of tasks at the barrier */
int br_n_threads; /* num of tasks participating in the barrier
synchronization */
} barrier_t;
barrier(barrier_t *br)
{
mutex_lock(&br->br_lock);
br->br_count++;
if (br->br_count < br->br_n_threads)
      cond_wait(&br->br_cond, &br->br_lock);
```

```
else
{
br->br_count = 0;
cond_broadcast(&br->br_cond);
}
      mutex_unlock(&br->br_lock);
      }
```

- Each participating task invokes the function barrier for barrier synchronization. The guarding mutex for br_count and br_n_threads is acquired on line #2. The number of waiting tasks at the barrier is updated on line #3.
- Line #4 checks to see if all of the participating tasks have reached the barrier. If more tasks are to arrive, the caller waits at the barrier (the blocking wait on the condition variable at line #5).
- If the caller is the last task of the group to enter the barrier, this task resets the barrier on line #6 and notifies all other tasks that the barrier synchronization is complete. Broadcasting on the condition variable on line #7 completes the barrier synchronization.

COMMUNICATION:

- Tasks communicate with one another so that they can pass information to each other and coordinate their activities in a multithreaded embedded application.
- Communication can be signal-centric, data-centric, or both. In signal-centric communication, all necessary information is conveyed within the event signal itself.
- In data-centric communication , information is carried within the transferred data. When the two are combined, data transfer accompanies event notification.

- When communication involves data flow and is unidirectional, this communication model is called loosely coupled communication. In this model, the data producer does not require a response from the consumer; the figure illustrates an example of loosely coupled communication.

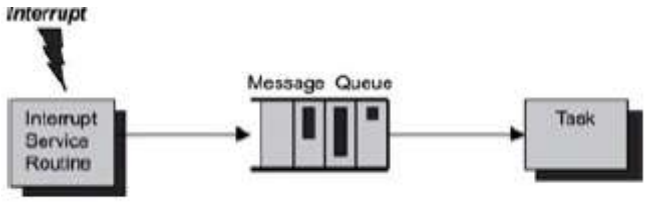


Figure : Loosely coupled ISR-to-task communication using message queues.

- For example, an ISR for an I/O device retrieves data from a device and routes the data to a dedicated processing task. The ISR neither solicits nor requires feedback from the processing task. By contrast, in tightly coupled communication, the data movement is bidirectional.
- The data producer synchronously waits for a response to its data transfer before resuming execution, or the response is returned asynchronously while the data producer continues its function.

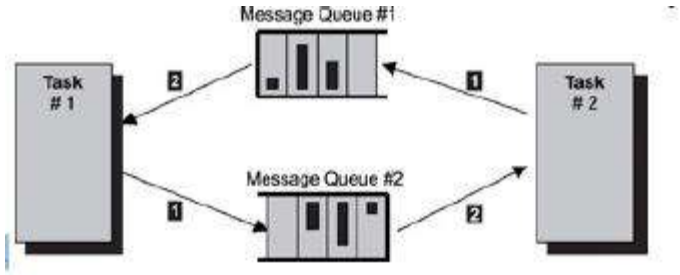


Figure : Tightly coupled task-to-task communication using message queues.

- In tightly coupled communication, as shown in Figure, task #1 sends data to task #2 using message queue #2 and waits for confirmation to arrive at message queue #1. The data communication is bidirectional.
- It is necessary to use a message queue for confirmations because the confirmation should contain enough information in case task #1 needs to re-send the data.
- Task #1 can send multiple messages to task #2, i.e., task #1 can continue sending messages while waiting for confirmation to arrive on message queue #2.
- Communication has several purposes, including the following:
  - transferring data from one task to another,

- signaling the occurrences of events between tasks,
- allowing one task to control the execution of other tasks,
- synchronizing activities, and
- implementing custom synchronization protocols for resource sharing.

- The first purpose of communication is for one task to transfer data to another task. Between the tasks, there can exist data dependency, in which one task is the data producer and another task is the data consumer.
- For example, consider a specialized processing task that is waiting for data to arrive from message queues or pipes or from shared memory.
- In this case, the data producer can be either an ISR or another task. The consumer is the processing task. The data source can be an I/O device or another task.
- The second purpose of communication is for one task to signal the occurrences of events to another task. Either physical devices or other tasks can generate events. A task or an ISR that is responsible for an event, such as an I/O event, or a set of events can signal the occurrences of these events to other tasks. Data might or might not accompany event signals.
- Consider, for example, a timer chip ISR that notifies another task of the passing of a time tick. The third purpose of communication is for one task to control the execution of other tasks. Tasks can have a master/slave relationship, known as *process control.* For example, in a control system, a master task that has the full knowledge of the entire running system controls individual subordinate tasks.
- Each subtask is responsible for a component, such as various sensors of the control system. The master task sends commands to the subordinate tasks to enable or disable sensors. In this scenario, data flow can be either unidirectional or bidirectional if feedback is returned from the subordinate tasks.
- The fourth purpose of communication is to synchronize activities. The fifth purpose of communication is to implement additional synchronization protocols for resource sharing. The tasks of a multithreaded program can implement custom, more-complex resource synchronization protocols on top of the system-supplied synchronization primitives.

CONCURRENCY:

- Computer science defines concurrency as a property of systems where several processes are executing at the same time, and may or may not interact with each other.
- In a single processor, concurrent external activities must be ”mapped” as pseudo-concurrent sequential processes.
- The timing requirements is met when all the sequential processes can react within the given deadlines. This may be difficult to prove for hard Real-Time systems.

- True concurrency requires parallel processing in separate processors, either a multi-processor system
- Whatever the solution, process-process interaction (communication) greatly complicates the implementation and analysis of a concurrent system.

REAL TIME - CONCURRENT SYSTEMS:

- A Real-Time system is any information processing system which has to respond to externally generated input stimuli within a finite and specified period, and the correctness depends not only on the logical result but also the time it was delivered
- A dedicated (embedded) system using a micro-controller to read out data at a fixed rate or from interrupts is a Real Time system, but is not a concurrent system if all processing is done within the same process Vice versa: a concurrent system where the correctness does not really depend on timing constraints, is not a Real Time system.
- However, in general a Real Time system is a concurrent system, where each Real Time activity is mapped into a process

CONCURRENT PROGRAMMING:

- The name given to programming notation and techniques for expressing potential parallelism and solving the resulting synchronization and communication problems
- For Real-Time systems the physical activities are mapped into a number of separately executing programs.
- All Real-Time systems are inherently concurrent — physical devices operate in parallel in the real world
- An alternative is to use sequential programming techniques . The programmer must construct the system so that it involves the cyclic execution of a program sequence to handle the various concurrent activities.

DEFINING SEMAPHORES:

- A *semaphore* (sometimes called a *semaphore token*) is a kernel object that one or more threads of execution can acquire or release for the purposes of synchronization or mutual exclusion.
- When a semaphore is first created, the kernel assigns to it an associated semaphore control block (SCB), a unique ID, a value (binary or a count), and a task-waiting list, as shown in Figure.

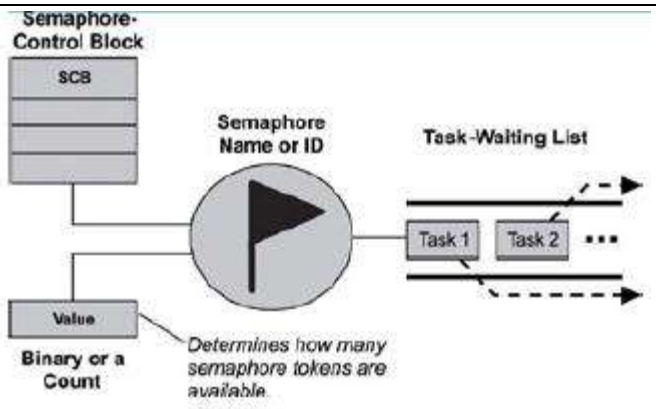


Figure : A semaphore, its associated parameters, and supporting data structures

- A semaphore is like a key that allows a task to carry out some operation or to access a resource. If the task can acquire the semaphore, it can carry out the intended operation or access the resource.
- A single semaphore can be acquired a finite number of times. In this sense, acquiring a semaphore is like acquiring the duplicate of a key from an apartment manager when the apartment manager runs out of duplicates, the manager can give out no more keys.
- Likewise, when a semaphore s limit is reached, it can no longer be acquired until someone gives a key back or releases the semaphore.
- The kernel tracks the number of times a semaphore has been acquired or released by maintaining a token count, which is initialized to a value when the semaphore is created. As a task acquires the semaphore, the token count is decremented; as a task releases the semaphore, the count is incremented.
- If the token count reaches 0, the semaphore has no tokens left. A requesting task, therefore, cannot acquire the semaphore, and the task blocks if it chooses to wait for the semaphore to become available.
- The task-waiting list tracks all tasks blocked while waiting on an unavailable semaphore. These blocked tasks are kept in the task-waiting list in either first in/first out (FIFO) order or highest priority first order.
- When an unavailable semaphore becomes available, the kernel allows the first task in the task-waiting list to acquire it. The kernel moves this unblocked task either to the running state, if it is the highest priority task, or to the ready state, until it becomes the highest priority task and is able to run. Note that the exact implementation of a task-waiting list can vary from one kernel to another.
- A kernel can support many different types of semaphores, including binary, counting, and mutual-exclusion (mutex) semaphores.

BINARY SEMAPHORES:

- A *binary semaphore* can have a value of either 0 or 1. When a binary semaphore s value is 0, the semaphore is considered *unavailable* (or *empty);* when the value is 1, the binary semaphore is considered *available* (or *full*). Note that when a binary semaphore is first created, it can be initialized to either available or unavailable (1 or 0, respectively). The state diagram of a binary semaphore is shown in Figure .

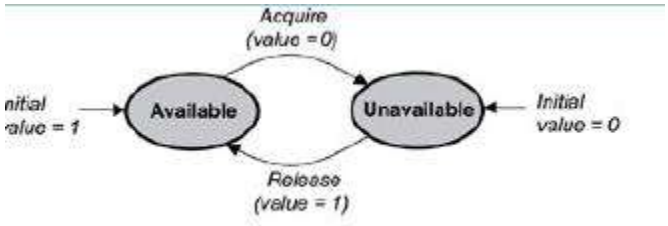


Figure : The state diagram of a binary semaphore.

- Binary semaphores are treated as global resources, which mean they are shared among all tasks that need them. Making the semaphore a global resource allows any task to release it, even if the task did not initially acquire it.

COUNTING SEMAPHORES:

- A counting semaphore uses a count to allow it to be acquired or released multiple times. When creating a counting semaphore, assign the semaphore a count that denotes the number of semaphore tokens it has initially.
- If the initial count is 0, the counting semaphore is created in the unavailable state. If the count is greater than 0, the semaphore is created in the available state, and the number of tokens it has equals its count, as shown in figure.

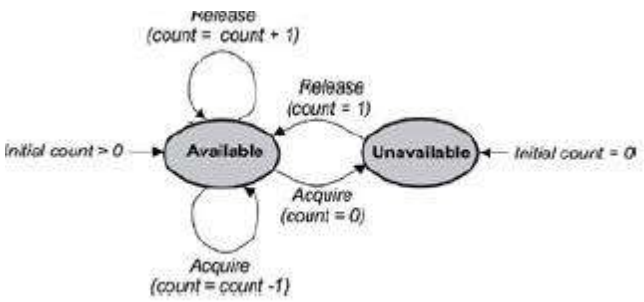


Figure : The state diagram of a counting semaphore.

- One or more tasks can continue to acquire a token from the counting semaphore until no tokens are left. When all the tokens are gone, the count equals 0, and the counting semaphore moves from the available state to the unavailable state. To move from the unavailable state back to the available state, a semaphore token must be released by any task.

MUTUAL EXCLUSION (MUTEX) SEMAPHORES:

- *A mutual exclusion (mutex) semaphore* is a special binary semaphore that supports ownership, recursive access, task deletion safety, and one or more protocols for avoiding problems inherent to mutual exclusion. Figure illustrates the state diagram of a mutex.

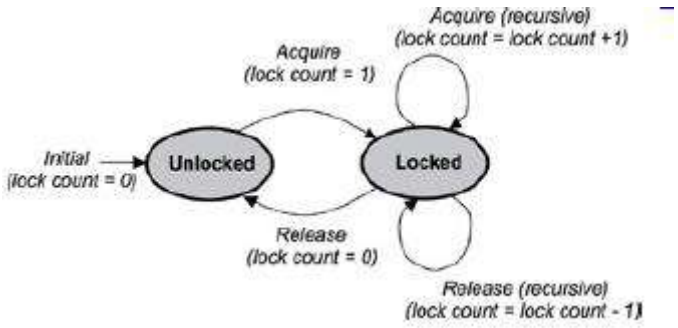


Figure : The state diagram of a mutual exclusion (mutex) semaphore.

- As opposed to the available and unavailable states in binary and counting semaphores, the states of a mutex are *unlocked* or *locked* (0 or 1, respectively). A mutex is initially created in the unlocked state, in which it can be acquired by a task. After being acquired, the mutex moves to the locked state.
- Conversely, when the task releases the mutex, the mutex returns to the unlocked state. Note that some kernels might use the terms *lock* and *unlock* for a mutex instead of *acquire* and *release*.

MUTEX OWNERSHIP:

- *Ownership* of a mutex is gained when a task first locks the mutex by acquiring it. Conversely, a task loses ownership of the mutex when it unlocks it by releasing it.
- When a task owns the mutex, it is not possible for any other task to lock or unlock that mutex.
- Contrast this concept with the binary semaphore, which can be released by any task, even a task that did not originally acquire the semaphore.

RECURSIVE LOCKING:

- Many mutex implementations also support recursive locking, which allows the task that owns the mutex to acquire it multiple times in the locked state. Depending on the implementation, recursion within a mutex can be automatically built into the mutex, or it might need to be enabled explicitly when the mutex is first created.
- The mutex with recursive locking is called a recursive mutex. This type of mutex is most useful when a task requiring exclusive access to a shared resource calls one or more routines that also require access to the same resource.
- A recursive mutex allows nested attempts to lock the mutex to succeed, rather than cause deadlock, which is a condition in which two or more tasks are blocked and are waiting on mutually locked resources.
- As shown in figure, when a recursive mutex is first locked, the kernel registers the task that locked it as the owner of the mutex. On successive attempts, the kernel uses an internal lock count associated with the mutex to track the number of times that the task currently owning the mutex has recursively acquired it. To properly unlock the mutex, it must be released the same number of times.

TASK DELETION SAFETY:

- Some mutex implementations also have built-in task deletion safety. Premature task deletion is avoided by using task deletion locks when a task locks and unlocks a mutex. Enabling this capability within a mutex ensures that while a task owns the mutex, the task cannot be deleted.
- Typically protection from premature deletion is enabled by setting the appropriate initialization options when creating the mutex.

PRIORITY INVERSION AVOIDANCE:

- Priority inversion commonly happens in poorly designed real-time embedded applications. Priority inversion occurs when a higher priority task is blocked and is waiting for a resource being used by a lower priority task, which has itself been preempted by an unrelated medium-priority task.
- In this situation, the higher priority task s priority level has effectively been inverted to the lower priority task s level.
- Enabling certain protocols that are typically built into murexes can help avoid priority inversion. Two common protocols used for avoiding priority inversion include:

- **Priority inheritance protocol** ensures that the priority level of the lower priority task that has acquired the mutex is raised to that of the higher priority task that has requested the mutex when inversion happens. Thepriority of the raised task is lowered to its original value after the task releases the mutex that the higher priority task requires.

- **Ceiling priority protocol** ensures that the priority level of the task that acquires the mutex is automatically set to the highest priority of all possible tasks that might request that mutex when it is first acquired until it is released.

SEMAPHORE OPERATIONS AND USE:

- Typical operations that developers might want to perform with the semaphores in an application include:
- creating and deleting semaphores,
- acquiring and releasing semaphores,
- clearing a semaphore s task-waiting list, and
- getting semaphore information.

CREATING AND DELETING SEMAPHORES:

Operation	Description
Create	Creates a semaphore
Delete	Deletes a semaphore

Table : Semaphore creation and deletion operations.

Several things must be considered, however, when creating and deleting semaphores. If a kernel supports different types of semaphores, different calls might be used for creating binary, counting, and mutex semaphores, as follows:

- **Binary** specify the initial semaphore state and the task-waiting order.
- **Counting** specify the initial semaphore count and the task-waiting order.
- **Mutex** specify the task-waiting order and enable task deletion safety, recursion, and priority-inversion avoidance protocols, if supported.

- Semaphores can be deleted from within any task by specifying their IDs and making semaphore-deletion calls.
- Deleting a semaphore is not the same as releasing it. When a semaphore is deleted, blocked tasks in its task-waiting list are unblocked and moved either to the ready state or to the running state (if the unblocked task has the highest priority). Any tasks, however, that try to acquire the deleted semaphore return with an error because the semaphore no longer exists.

ACQUIRING AND RELEASING SEMAPHORES:

Operation	Description
Acquire	Acquire a semaphore token
Release	Release a semaphore token

Table : Semaphore acquire and release operations.

- The operations for acquiring and releasing a semaphore might have different names, depending on the kernel: for example, *take* and *give* , *sm_p* and *sm_v* , *pend* and *post* , and *lock* and *unlock* . Regardless of the name, they all effectively acquire and release semaphores.
- Tasks typically make a request to acquire a semaphore in one of the following ways:

➢ **Wait forever** task remains blocked until it is able to acquire a semaphore.

➢ **Wait with a timeout** task remains blocked until it is able to acquire a semaphore or until a set interval of time, called the *timeout interval* , passes. At this point, the task is removed from the semaphore stask-waiting list and put in either the ready state or the running state.

➢ **Do not wait** task makes a request to acquire a semaphore token, but, if one is not available, the task does not block.

```
broadcastTask ()
                :
                Send broadcast message to queue
                :
}

Note: similar code for tSignalTasks 1, 2, and 3.

tSignalTask ()
{
                :
                Receive message on queue
                :
}
```

CLEARING SEMAPHORE TASK-WAITING LISTS:

➢ To clear all tasks waiting on a semaphore task-waiting list, some kernels support a *flush* operation, as shown in Table:

Operation	Description
Flush	Unblocks all tasks waiting on a semaphore

Table : Semaphore information operations.

- The flush operation is useful for broadcast signaling to a group of tasks. For example, a developer might design multiple tasks to complete certain activities first and then block while trying to acquire a common semaphore that ismade unavailable.

- After the last task finishes doing what it needs to, the task can execute a semaphore flush operation on the common semaphore. This operation frees all tasks waiting in the semaphore s task waiting list.
- The synchronization scenario just described is also called *thread rendezvous,* when multiple tasks executions need to meet at some point in time to synchronize execution control.

GETTING SEMAPHORE INFORMATION:

Operation	Description
Show info	Show general information about semaphore
Show blocked tasks	Get a list of IDs of tasks that are blocked on a semaphore

Table: Semaphore information operations

- These operations are relatively straightforward but should be used judiciously, as the semaphore information might be dynamic at the time it is requested.

SEMAPHORE USE:

- Semaphores are useful either for synchronizing execution of multiple tasks or for coordinating access to a shared resource. The following examples and general discussions illustrate using different types of semaphores to address common synchronization design requirements effectively, as listed:

- wait-and-signal synchronization,
- multiple-task wait-and-signal synchronization,
- credit-tracking synchronization,
- single shared-resource-access synchronization,
- recursive shared-resource-access synchronization, and
- multiple shared-resource-access synchronization.

WAIT-AND-SIGNAL SYNCHRONIZATION:

- Two tasks can communicate for the purpose of synchronization without exchanging data. For example, a binary semaphore can be used between two tasks to coordinate the transfer of execution control, as shown in Figure.

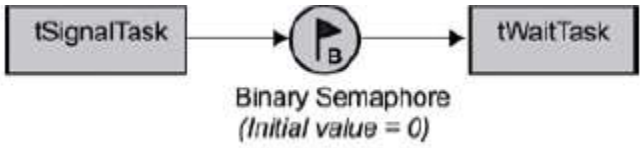


Figure : Wait-and-signal synchronization between two tasks.

- The task makes a request to acquire the semaphore but is blocked because the semaphore is unavailable. This step gives the lower priority tSignalTask a chance to run; at some point, tSignalTask releases the binary semaphore and unblocks tWaitTask. The pseudo code for this scenario is shown in listing..

```
      tWaitTask ( )
{
:
Acquire binary semaphore token
:
}
tSignalTask ( )
{
:
Release binary semaphore token
:
}
```

- Because tWaitTask's priority is higher than tSignalTask's priority, as soon as the semaphore is released, tWaitTask preempts tSignalTask and starts to execute.

MULTIPLE-TASK WAIT-AND-SIGNAL SYNCHRONIZATION:

- When coordinating the synchronization of more than two tasks, use the flush operation on the task-waiting list of a binary semaphore, as shown in Figure

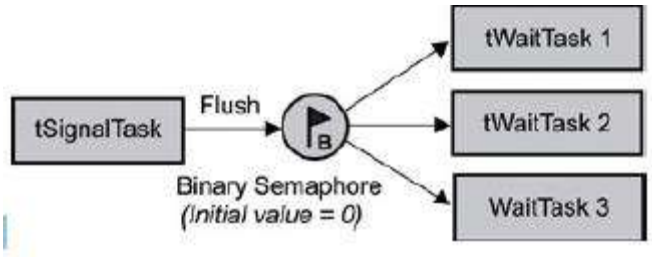


Figure 6: Wait-and-signal synchronization between multiple tasks.

CREDIT-TRACKING SYNCHRONIZATION:

- Sometimes the rate at which the signaling task executes is higher than that of the signaled task. In this case, a mechanism is needed to count each signaling occurrence. The counting semaphore provides just this facility.
- With a counting semaphore, the signaling task can continue to execute and increment a count at its own pace, while the wait task, when unblocked, executes at its own pace, as shown in Figure

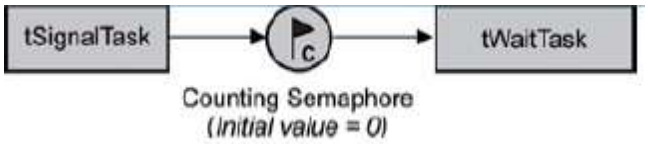


Figure : Credit-tracking synchronization between two tasks.

- Again, the counting semaphore's count is initially 0, making it unavailable. The lower priority tWaitTask tries to acquire this semaphore but blocks until tSignalTask makes the semaphore available by performing a release on it.
- Even then, tWaitTask will waits in the ready state until the higher priority tSignalTask eventually relinquishes the CPU by making a blocking call or delaying itself, as shown in Listing.

```
      tWaitTask ()
{
:
Acquire counting semaphore token
:
}
tSignalTask ()
{
:
Release counting semaphore token
```

}

Listing : Pseudo code for credit-tracking synchronization.

SINGLE SHARED-RESOURCE-ACCESS SYNCHRONIZATION:

- One of the more common uses of semaphores is to provide for mutually exclusive access to a shared resource. A shared resource might be a memory location, a data structure, or an I/O device-essentially anything that might have to be shared between two or more concurrent threads of execution.
- A semaphore can be used to serialize access to a shared resource, as shown in Figure ..

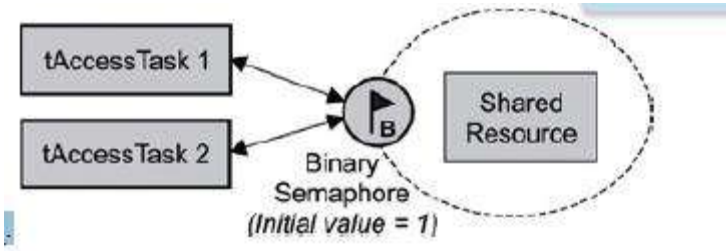


Figure : Single shared-resource-access synchronization.

- In this scenario, a binary semaphore is initially created in the available state (value = 1) and is used to protect the shared resource.
- To access the shared resource, task 1 or 2 needs to first successfully acquire the binary semaphore before reading from or writing to the shared resource. The pseudo code for both tAccessTask 1 and 2 is similar to Listing.

```
tAccessTask ()
{
:
Acquire binary semaphore token
Read or write to shared resource
      Release binary semaphore token

}
```

SINGLE SHARED-RESOURCE-ACCESS SYNCHRONIZATION:

- One of the more common uses of semaphores is to provide for mutually exclusive access to a shared resource. A shared resource might be a memory location, a data structure, or an I/O device-essentially anything that might have to be shared between two or more concurrent threads

of execution. A semaphore can be used to serialize access to a shared resource, as shown in Figure.

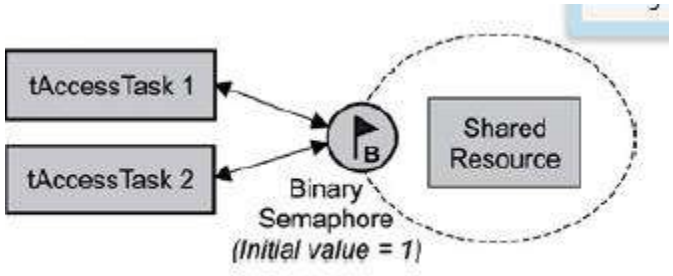


Figure : Single shared-resource-access synchronization.

- In this scenario, a binary semaphore is initially created in the available state (value = 1) and is used to protect the shared resource. To access the shared resource, task 1 or 2 needs to first successfully acquire the binary semaphore before reading from or writing to the shared resource. The pseudo code for both tAccessTask 1 and 2 is similar to Listing .

```
tAccessTask ()
{
:
Acquire binary semaphore token
Read or write to shared resource
Release binary semaphore token
:
      }
```

RECURSIVE SHARED-RESOURCE-ACCESS SYNCHRONIZATION:

- Sometimes a developer might want a task to access a shared resource recursively. This situation might exist if tAccessTask calls Routine A that calls Routine B, and all three need access to the same shared resource, as shown in figure.

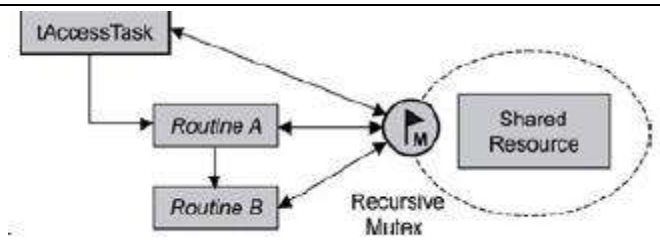


Figure: Recursive shared- resource-access synchronization.

- If a semaphore were used in this scenario, the task would end up blocking, causing a deadlock. When a routine is called from a task, the routine effectively becomes a part of the task. When Routine A runs, therefore, it is running as a part of tAccessTask
- . Routine A trying to acquire the semaphore is effectively the same as tAccessTask trying to acquire the same semaphore. In this case, tAccessTask would end up blocking while waiting for the unavailable semaphore that it already has. One solution to this situation is to use a recursive mutex.
- After tAccessTask locks the mutex, the task owns it. Additional attempts from the task itself or from routines that it calls to lock the mutex succeed. As a result, when Routines A and B attempt to lock the mutex, they succeed without blocking. The pseudo code for tAccessTask, Routine A, and Routine B are similar to Listing .

```
tAccessTask ()
{
:
Acquire mutex
Access shared resource
Call Routine A
Release mutex
:
}
Routine A ()
{
:
Acquire mutex
Access shared resource
Call Routine B
Release mutex
:
}
Routine B ()
```

```
{
:
Acquire mutex
Access shared resource
      Release mutex
}
```

MULTIPLE SHARED-RESOURCE-ACCESS SYNCHRONIZATION:

- For cases in which multiple equivalent shared resources are used, a counting semaphore comes in handy, as shown in

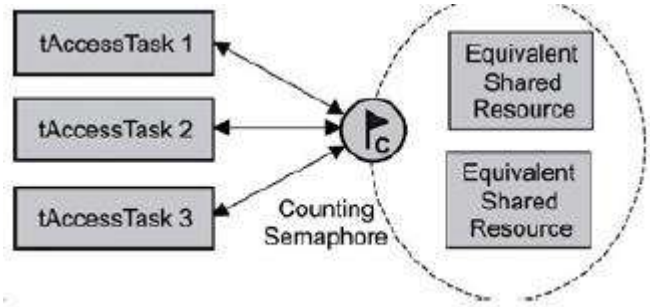


Figure : Single shared-resource-access synchronization.

- Note that this scenario does not work if the shared resources are not equivalent. The counting semaphore's count is initially set to the number of equivalent shared resources: in this example, 2. As a result, the first two tasks requesting a semaphore token are successful.
- However, the third task ends up blocking until one of the previous two tasks releases a semaphore token, as shown in Listing . Note that similar code is used for tAccessTask 1, 2, and 3.

```
tAccessTask ()
{
:
Acquire a counting semaphore token
Read or Write to shared resource
Release a counting semaphore token
:
            }
```

Listing : Pseudo code for multiple tasks accessing equivalent shared resources

DEFINING MESSAGE QUEUE:

- A message queue is a buffer-like object through which tasks and ISRs send and receive messages to communicate and synchronize with data. A message queue is like a pipeline. It temporarily holds messages from a sender until the intended receiver is ready to read them. This temporary buffering decouples a sending and receiving task; that is, it frees the tasks from having to send and receive messages simultaneously.
- A message queue has several associated components that the kernel uses to manage the queue. When a message queue is first created, it is assigned an associated queue control block (QCB), a message queue name, a unique ID, memory buffers, a queue length, a maximum message length, and one or more task-waiting lists, as illustrated in figure.

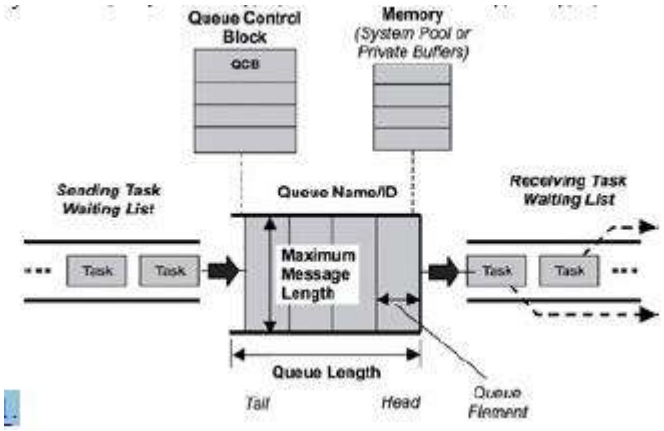


Figure : A message queue, its associated parameters, and supporting data structures.

- It is the kernel s job to assign a unique ID to a message queue and to create its QCB and task-waiting list. The kernel also takes developer-supplied parameters such as the length of the queue and the maximum message length to determine how much memory is required for the message queue.
- After the kernel has this information, it allocates memory for the message queue from either a pool of system memory or some private memory space.
- The message queue itself consists of a number of elements, each of which can hold a single message. The elements holding the first and last messages are called the head and tail respectively. Some elements of the queue may be empty (not containing a message). The total

number of elements (empty or not) in the queue is the total length of the queue . The developer specified the queue length when the queue was created.

MESSAGE QUEUE STATES:

- As with other kernel objects, message queues follow the logic of a simple FSM, as shown in Figure. When a message queue is first created, the FSM is in the empty state. If a task attempts to receive messages from this message queue while the queue is empty, the task blocks and, if it chooses to, is held on the message queue's task-waiting list, in either a FIFO or priority-based order.

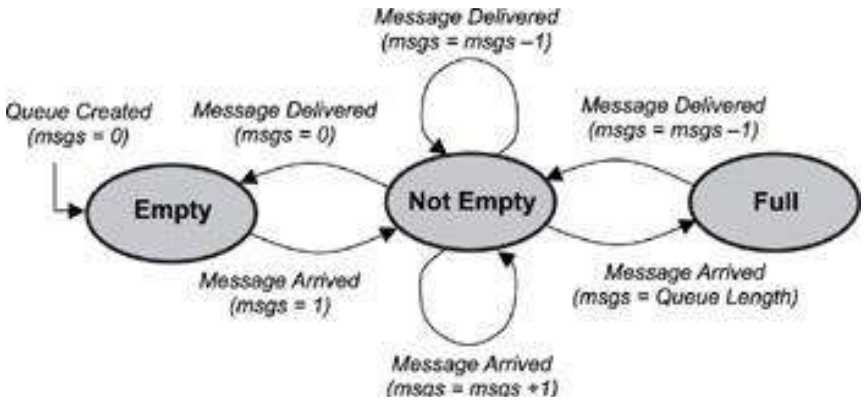


Figure : The state diagram for a message queue.

- In this scenario, if another task sends a message to the message queue, the message is delivered directly to the blocked task. The blocked task is then removed from the task-waiting list and moved to either the ready or the running state. The message queue i n this case remains empty because it has successfully delivered the message.
- If another message is sent to the same message queue and no tasks are waiting in the message queue's task-waiting list, the message queue's state becomes not empty.
- As additional messages arrive at the queue, the queue eventually fills up until it has exhausted its free space. At this point, the number of messages in the queue is equal to the queue's length, and the message queue's state becomes full.
- While a message queue is in this state, any task sending messages to it will not be successful unless some other task first requests a message from that queue, thus freeing a queue element.
- In some kernel implementations when a task attempts to send a message to a full message queue, the sending function returns an error code to that task. Other kernel implementations allow such a task to block, moving the blocked task into the sending task-waiting list, which is separate from the receiving task-waiting list.

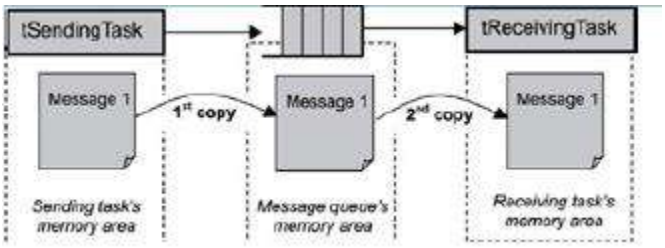


Figure: Message copying and memory use for sending and receiving messages.

MESSAGE QUEUE CONTENT:

Message queues can be used to send and receive a variety of data. Some examples include:

- a temperature value from a sensor,
- a bitmap to draw on a display,
- a text message to print to an LCD,
- a keyboard event, and
- a data packet to send over the network.

- Some of these messages can be quite long and may exceed the maximum message length, which is determined when the queue is created. (Maximum message length should not be confused with total queue length, which is the total number of messages the queue can hold.)
- One way to overcome the limit on message length is to send a pointer to the data, rather than the data itself. Even if a long message might fit into the queue, it is sometimes better to send a pointer instead in order to improve both performance and memory utilization.
- When a task sends a message to another task, the message normally is copied twice, as shown in Figure .The first time, the message is copied when the message is sent from the sending task s memory area to the message queue s memory area.
- The second copy occurs when the message is copied from the message queue s memory area to the receiving task s memory area. An exception to this situation is if the receiving task is already blocked waiting at the message queue. Depending on a kernel s implementation, the message might be copied just once in this case from the sending task s memory area to the receiving task s memory area, bypassing the copy to the message queue s memory area.

- Because copying data can be expensive in terms of performance and memory requirements, keep copying to a minimum in a real-time embedded system by keeping messages small or, if that is not feasible, by using a pointer instead.

MESSAGE QUEUE STORAGE:

- Different kernels store message queues in different locations in memory. One kernel might use a system pool, in which the messages of all queues are stored in one large shared area of memory. Another kernel might use separate memory areas, called private buffers, for each message queue.

SYSTEM POOLS:

- Using a system pool can be advantageous if it is certain that all message queues will never be filled to capacity at the same time. The advantage occurs because system pools typically save on memory use.
- The downside is that a message queue with large messages can easily use most of the pooled memory, not leaving enough memory for other message queues.
- Indications that this problem is occurring include a message queue that is not full that starts rejecting messages sent to it or a full message queue that continues to accept more messages.

PRIVATE BUFFERS:

- Using private buffers, on the other hand, requires enough reserved memory area for the full capacity of every message queue that will be created.
- This approach clearly uses up more memory; however, it also ensures that messages do not get overwritten and that room is available for all messages, resulting in better reliability than the pool approach.

OPERATIONS AND USE:

MESSAGE QUEUE:

- Typical message queue operations include the following:

  - creating and deleting message queues,
  - sending and receiving messages, and
  - obtaining message queue information.

CREATING AND DELETING MESSAGE QUEUES:

Message queues can be created and deleted by using two simple calls, as shown in Table.

Operation	Description
Create	Creates a message queue
Delete	Deletes a message queue

Table : Message queue creation and deletion operations.

- When created, message queues are treated as global objects and are not owned by any particular task. Typically, the queue to be used by each group of tasks or ISRs is assigned in the design.
- When creating a message queue, a developer needs to make some initial decisions about the length of the message queue, the maximum size of the messages it can handle, and the waiting order for tasks when they block on a message queue.
- Deleting a message queue automatically unblocks waiting tasks. The blocking call in each of these tasks returns with an error. Messages that were queued are lost when the queue is deleted.

SENDING AND RECEIVING MESSAGES:

- The most common uses for a message queue are sending and receiving messages. These operations are performed in different ways, some of which are listed in Table .

Operation	Description
Send	Sends a message to a message queue
Receive	Receives a message from a message queue
Broadcast	Broadcasts messages

Table : Sending and receiving messages

SENDING MESSAGES:

- When sending messages, a kernel typically fills a message queue from head to tail in FIFO order, as shown in figure each new message is placed at the end of the queue

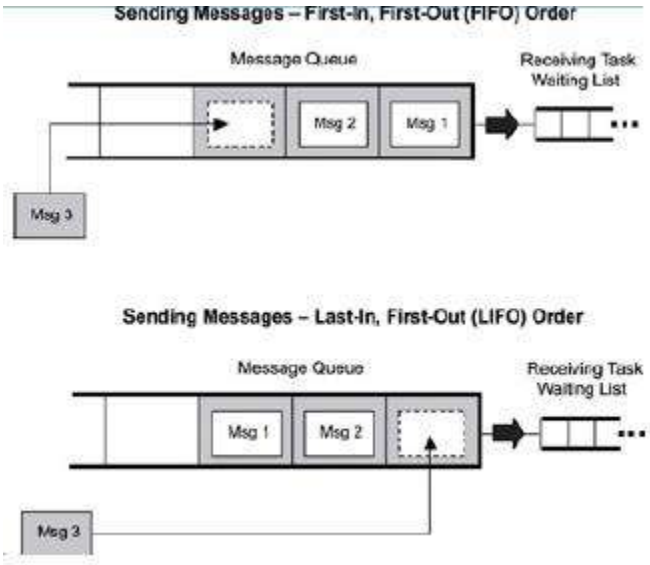


Figure : Sending messages in FIFO or LIFO order.

- In the case of a task set to block forever when sending a message, the task blocks until a message queue element becomes free (e.g., a receiving task takes a message out of the queue).
- In the case of a task set to block for a specified time, the task is unblocked if either a queue element becomes free or the timeout expires, in which case an error is returned.

RECEIVING MESSAGES:

- As with sending messages, tasks can receive messages with different blocking policies the same way as they send them with a policy of not blocking, blocking with a timeout, or blocking forever. Note, however, that in this case, the blocking occurs due to the message queue being empty, and the receiving tasks wait in either a FIFO or priority based order.
- The diagram for the receiving tasks is similar to figure , except that the blocked receiving tasks are what fills the task list.
- Messages can be read from the head of a message queue in two different ways:

  - destructive read, and
  - non-destructive read.

OBTAINING MESSAGE QUEUE INFORMATION:

- Obtaining message queue information can be done from an application by using the operations listed in Table.

Operation	Description
Show queue info	Gets information on a message queue
Show queue s task-waiting list	Gets a list of tasks in the queue s task-waiting list

Table : Obtaining message queue information operations

- Different kernels allow developers to obtain different types of information about a message queue, including the message queue ID, the queuing order used for blocked tasks (FIFO or priority-based), and the number of messages queued.
- Some calls might even allow developers to get a full list of messages that have been queued up. As with other calls that get information about a particular kernel object, be careful when using these calls.
- The information is dynamic and might have changed by the time it s viewed. These types of calls should only be used for debugging purposes

MESSAGE QUEUE USE:

The following are typical ways to use message queues within an application:

- non-interlocked, one-way data communication,
- interlocked, one-way data communication,
- interlocked, two-way data communication, and
- broadcast communication.

NON-INTERLOCKED, ONE-WAY DATA COMMUNICATION:

- One of the simplest scenarios for message-based communications requires a sending task (also called the message source), a message queue, and a receiving task (also called a message sink), as illustrated in Figure.

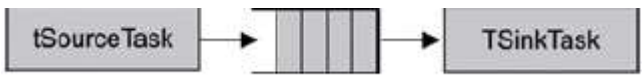


Figure : Non-interlocked, one-way data communication.

- This type of communication is also called non-interlocked (or loosely coupled), one-way data communication. The activities of tSourceTask and tSinkTask are not synchronized. TSourceTask simply sends a message; it does not require acknowledgement from tSinkTask.
- The pseudo code for this scenario is provided in Listing

```
ceTask ()

                :
                Send message to message queue
                :
}

tSinkTask ()
{
                :
                Receive message from message queue
                :
}
```

Listing : Pseudo code for non-interlocked, one-way data communication

- If tSinkTask is set to a higher priority, it runs first until it blocks on an empty message queue. As soon as tSourceTask sends the message to the queue, tSinkTask receives the message and starts to execute again.
- If tSinkTask is set to a lower priority, tSourceTask fills the message queue with messages. Eventually, tSourceTask can be made to block when sending a message to a full message queue. These actions makes tSinkTask wake up and start taking messages out of the message queue.
- ISRs typically use non-interlocked, one-way communication. A task such as tSinkTask runs and waits on the message queue. When the hardware triggers an ISR to run, the ISR puts one or more messages into the message queue. After the ISR completes running, tSinkTask gets an opportunity to run (if it s the highest-priority task) and takes the messages out of the message queue.
- Remember, when ISRs send messages to the message queue, they must do so in a non-blocking way. If the message queue becomes full, any additional messages that the ISR sends to the message queue are lost.

INTERLOCKED, ONE-WAY DATA COMMUNICATION:

- In some designs, a sending task might require a handshake (acknowledgement) that the receiving task has been successful in receiving the message. This process is called interlocked

communication, in which the sending task sends a message and waits to see if the message is received.

- This requirement can be useful for reliable communications or task synchronization. For example, if the message for some reason is not received correctly, the sending task can resend it. Using interlocked communication can close a synchronization loop.
- To do so, it can construct a continuous loop in which sending and receiving tasks operate in lockstep with each other. An example of one-way, interlocked data communication is illustrated in Figure.

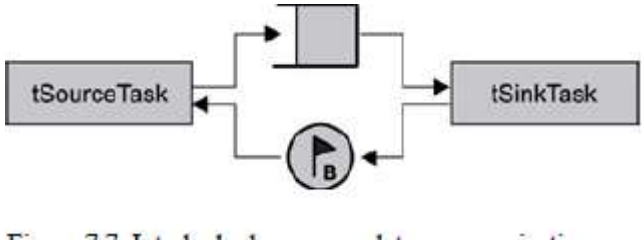


Figure : Interlocked, one-way data communication.

- In this case, tSourceTask and tSinkTask use a binary semaphore initially set to 0 and a message queue with a length of 1 (also called a mailbox). tSourceTask sends the message to the message queue and blocks on the binary semaphore. tSinkTask receives the message and increments the binary semaphore.
- The semaphore that has just been made available wakes up tSourceTask. tSourceTask, which executes and posts another message into the message queue, blocking again afterward on the binary semaphore.

INTERLOCKED, TWO-WAY DATA COMMUNICATION

- Sometimes data must flow bidirectional between tasks, which is called interlocked, two-way data communication (also called full-duplex or tightly coupled communication). This form of communication can be useful when designing a client/server-based system. A diagram is provided in Figure.

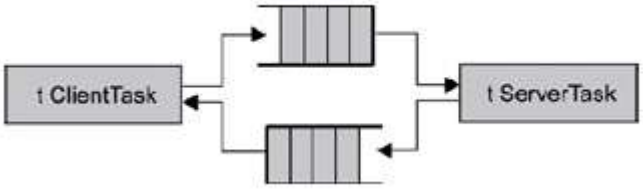


Figure : Interlocked, two-way data communication.

```
ceTask ()
{
            :
            Send message to message queue
            Acquire binary semaphore
            :
}

tSinkTask ()
{
            :
            Receive message from message queue
            Give binary semaphore
            :
}
```

Listing : Pseudo code for interlocked, one-way data communication

- In this case, tClientTask sends a request to tServerTask via a message queue. tServer-Task fulfills that request by sending a message back to tClientTask.

The pseudo code is provided in Listing.

```
ntTask ()

            :
            Send a message to the requests queue
            Wait for message from the server queue
            :
}

tServerTask ()
{
            :
            Receive a message from the requests queue
            Send a message to the client queue
            :
}
```

Listing : Pseudo code for interlocked, two-way data communication.

BROADCAST COMMUNICATION:

- Some message-queue implementations allow developers to broadcast a copy of the same message to multiple tasks, as shown in Figure.

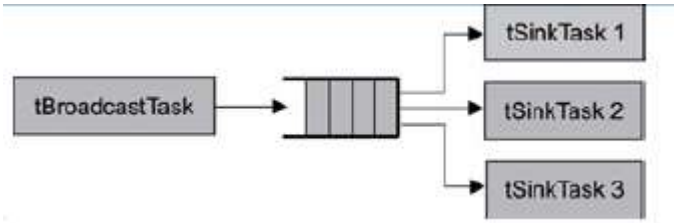


Figure : Broadcasting messages.

- Message broadcasting is a one-to-many-task relationship. tBroadcastTask sends the message on which multiple tSink-Task are waiting.
- Pseudo code for broadcasting messages is provided in Listing.

```
tBroadcastTask ()
{
                :
                Send broadcast message to queue
                :
}

Note: similar code for tSignalTasks 1, 2, and 3.

tSignalTask ()
{
                :
                Receive message on queue
                :
}
```

Listing : Pseudo code for broadcasting messages.

- In this scenario, tSinkTask 1, 2, and 3 have all made calls to block on the broadcast message queue, waiting for a message. When tBroadcastTask executes, it sends one message to the message queue, resulting in all three waiting tasks exiting the blocked state.

MERITS AND DEMERITS OF PREEMPTIVE SCHEDULING

PREEMPTIVE SCHEDULING

A higher priority process can preempt a currently running low priority process to avoid priority inversion.

Tasks of the Scheduler

• To decide which process can use the CPU

• Once it has had a period of time then it is placed into a ready state and the next process allowed to run

Advantages:

- The main advantage is that they ensure fairness to all jobs, regardless of its priority and also provide quick response time depending on the CPU time the job needs.

Disadvantages:

- The disadvantage of this method is that we need to cater for race conditions as well as having the responsibility of scheduling the processes

- The main disadvantage is that it provides indefinite postponement to process.

ACCURATE TIME MANAGEMENT IN REAL TIME KERNEL

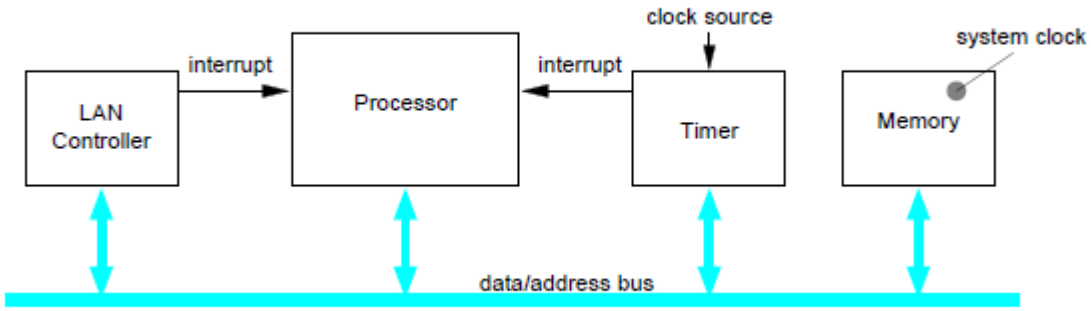
- A **real-time operating system** (**RTOS**) is an operating system (OS) intended to serve real-time applications which process data as it comes in, typically without buffering delays.
- Processing time requirements (including any OS delay) are measured in tenths of seconds or shorter increments of time. They either are event driven or time sharing.
- Event driven systems switch between tasks based on their priorities while time sharing systems switch the task based on clock interrupts.

- Accurate time management functions are required for scheduling real-time tasks to meet their deadlines.
- Time-based scheduling techniques make use of the worst-case execution time estimates of the tasks to generate deterministic schedules for hard real-time systems.
- With the advent of fast processors which can execute millions of instructions per second, considerable amount of computations can be done in a very short period of time.
- This in turn, demands accurate timing mechanisms for scheduling real-time tasks to achieve better processor utilization.
- A fast and accurate time keeping mechanism, such as a system clock with fine granularity is essential to implement such precise time-based scheduling algorithms

The following are the basic functionality required for such an accurate time management unit.

- A mechanism to implement a monotonic clock. The monotonicity of clock is very important because distributed applications assumes that time-stamps produced by a clock always increases monotonically
- An automatic mechanism to register the time of occurrence of user-defined events. This property is essential to accurately time-stamp events in a real-time system.
- A deterministic and atomic mechanism to read and update the system time.
- A hardware register to hold system time to the required resolution and a mechanism to increment the system time without any software intervention.
- A mechanism to compensate for the drift (internal and external) to maintain a consistent global time.

The heart of a real-time system is the scheduler that makes important decisions such as, which one of the ready tasks is to be scheduled next, when and how long each of these tasks are to be executed. The performance of such a real-time scheduler depends on it's capability to ensure that the real-time deadlines of the system are always satisfied, and to achieve high processor utilization at the same time. Most of the schedulers make use of some type of timeout mechanism to preempt the currently executing task and schedule the next task in the ready list, based on the scheduling algorithm used. Clearly, this timing mechanism should be very accurate, otherwise the tasks will be scheduled to run early or late and/or the task will be executed for longer or shorter amount of time than required. In other words, the correctness of the scheduling scheme directly depends on the timing mechanism used to implement it. Often the same timing mechanism is used to maintain the local (time-of-day or -year) clock of the system.



REQUIREMENTS IN THE SELECTION OF AN RTOS

RTOS Selection Ranking RTOS is both a tricky and difficult task because there are so many competent choices that are available in the market. The developer can choose from commercial RTOS (44% developers are using) or open- source RIOS (20 %) or internally developed RTOS (I7 %). This shows that almost 70% of developers are using the RTOS for their current projects [43] and are migrating from one RTOS to another due to various. reasons.

To handle the current requirements of the customers, developers are using 32 bit controllers in their projects; in which 92% projects/ products are using RTOS and 50% of developers are migrating to another RTOS for their next project. This influences importance of the selection of right RTOS for a particular project so that it meets all the requirements, and fulfills its intended task.

In order to select the RTOS, the designer first identifies the parameters for selection based on the application and the intended requirements are provided to the systems through an interactive GUI. The designer has the freedom to omit and or include parameters, and also he/she can edit the database of

RTOS for efficient selection under multi-user environment. Subsequently, genetic algorithm is used to arrive at the RTOS taking into account the parameters that are specified.

RTOS Parameters

Among the different parameters for selecting the RTOS, the significant ones are:

1. Interrupt Latency,
2. Context Switching,
3. Inter-task Communication (Message Queue Mechanism, Signal Mechanism and Semaphores),
4. Power Management (Sleep mode, Low power mode, Idle mode, and Stand by mode)
5. No. of Interrupt levels,
6. Kernel Size,
7. Scheduling Algorithms (Round Robin Scheduling, First Come First Serve, Shortest Job First, Preemptive Scheduling),
8. lnterrupt Levels,
9 Maintenance Fee,
10. Timers,
11. Priority Levels,
12. Kernel Synchronization (timers, mutexes, events, semaphores, etc),
13. Cost,
14. Development host,
15. Task Switching Time, and
16. Royalty Fee.

There are also other parameters like target processor support, Languages supported, Technical Support etc., are also important which are considered by the developer.

2 MARKS:

1. **What is an Operating system?**

   An operating system is a program that manages the computer hardware. It also provides a basis for application programs and act as an intermediary between a user of a computer and the computer hardware. It controls and coordinates the use of the hardware among the various application programs for the various users

2. What are the use of job queues, ready queues & device queues?

As a process enters a system, they are put into a job queue. This queue consists of all jobs in the system. The processes that are residing in main memory and are ready & waiting to execute are kept on a list called ready queue. The list of processes waiting for a particular I/O device is kept in the device queue.

3. What is meant by context switch? (NOV'14)

Switching the CPU to another process requires saving the state of the old process and loading the saved state for the new process. This task is known as context switch. The context of a process is represented in the PCB of a process.

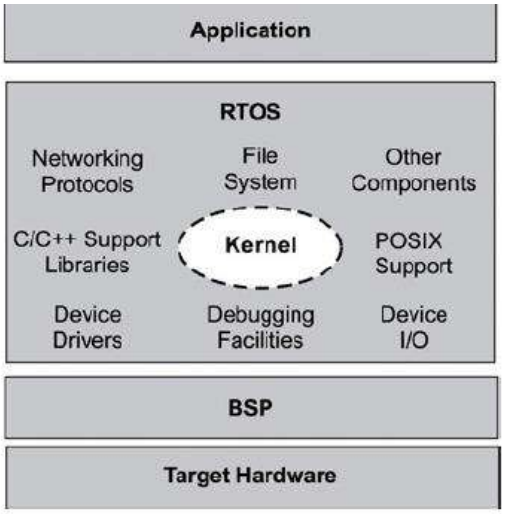
4. What is real time system?(APR 2011) (NOV 2011)

A real time system has well defined, fixed time constraints. Processing must be done within the defined constraints, or the system will fail. It is often used as a control device in a dedicated application.

5. What is scheduler?

A process migrates between the various scheduling queues throughout its life time. The OS must select processes from these queues in some fashion. This selection process is carried out by a scheduler.

6 .What is High-level view of an RTOS?



7. What are the functional similarities between a typical RTOS and GPOS ?

- some level of multitasking,
- software and hardware resource management,
- provision of underlying OS services to applications, and
- Abstracting the hardware from the software application.

8. Define busy waiting and spinlock?

When a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This is called as busy waiting and this type of semaphore is also called a spinlock, because the process while waiting for the lock.

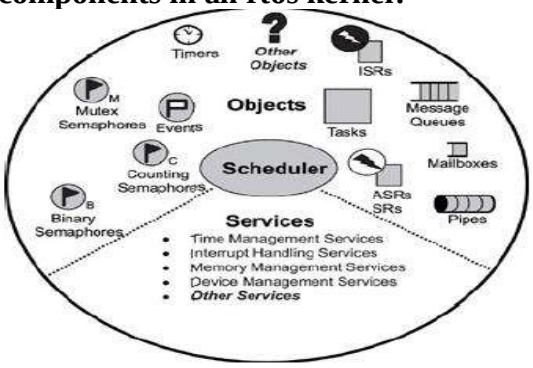
9. Write some key functional differences that set RTOSes apart from GPOSes.

- better reliability in embedded application contexts,
- the ability to scale up or down to meet application needs,
- faster performance,
- reduced memory requirements,
- scheduling policies tailored for real-time embedded systems,
- support for diskless embedded systems by allowing executables to boot and run from ROM or RAM, and
- better portability to different hardware platforms

10. Write down Scheduling Criteria?

- **CPU utilization** i.e. CPU usage - to maximize
- **Throughput** = number of processes that complete their execution per time unit – to maximize
- **Turnaround time** = amount of time to execute a particular process - to minimize
- **Waiting time** = amount of time a process has been waiting in the ready queue - to minimize
- **Response time** = amount of time it takes from when a job was submitted until it initiates its first response (output), **not** to time it completes output of its first response - to minimize

11. What are the common components in an rtos kernel?



12. What is the scheduler?

The scheduler is at the heart of every kernel. A scheduler provides the algorithms needed to determine which task executes when. To understand how scheduling works, this section describes the following topics:

- schedulable entities,
- multitasking,
- context switching,
- dispatcher, and
- Scheduling algorithms.

13. What is OBJECTS in RTOS?

- Kernel objects are special constructs that are the building blocks for application development for real-time embedded systems.
- The most common RTOS kernel objects are :
    - Tasks are concurrent and independent threads of execution that can compete for CPU execution time.
    - Semaphores are token-like objects that can be incremented or decremented by tasks for synchronization or mutual exclusion.

- Message Queues are buffer-like data structures that can be used for synchronization, mutual exclusion, and data exchange by passing messages between tasks.

14. What is services RTOS?

Most kernels provide services that help developers create applications for real-time embedded systems. These services comprise sets of API calls that can be used to perform operations on kernel objects or can be used in general to facilitate timer management, interrupt handling, device I/O, and memory management. Again, other services might be provided; these services are those most commonly found in RTOS kernels

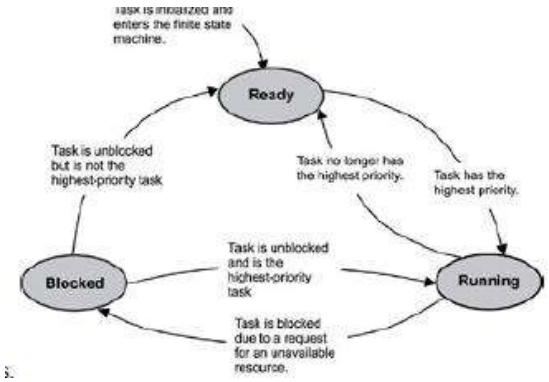
15. What are the CHARACTERISTICS OF AN RTOS? (Nov 2016) (Nov 2017)

- An application's requirements define the requirements of its underlying RTOS. Some of the more common attributes are:
- reliability,
- predictability,
- performance,
- compactness, and
- scalability.

16. Define A TASK?

A *task* is an independent thread of execution that can compete with other concurrent tasks for processor execution time. As mentioned earlier, developers decompose applications into multiple concurrent tasks to optimize the handling of inputs and outputs within set time constraints.

17. What is a typical finite state machine for task execution states.



18. What are the two types of SYNCHRONIZATION?

Synchronization is classified into two categories: *resource synchronization* and *activity synchronization* . Resource synchronization determines whether access to a shared resource is safe, and, if not, when it will be safe. Activity synchronization determines whether the execution of a multithreaded program has reached a certain state and, if it hasn't, how to wait for and be notified when this state is reached

19. Define Concurrency?

Computer science defines concurrency as a property of systems where several processes are executing at the same time, and may or may not interact with each other.

20. Defining Semaphores: (Nov 2016) (Nov 2017) (May 2017)

A *semaphore* (sometimes called a *semaphore token*) is a kernel object that one or more threads of execution can acquire or release for the purposes of synchronization or mutual exclusion.

21. Define Binary Semaphores?

A *binary semaphore* can have a value of either 0 or 1. When a binary semaphore s value is 0, the semaphore is considered *unavailable* (or *empty);* when the value is 1, the binary semaphore is considered *available* (or *full*).

22. Define mutual exclusion

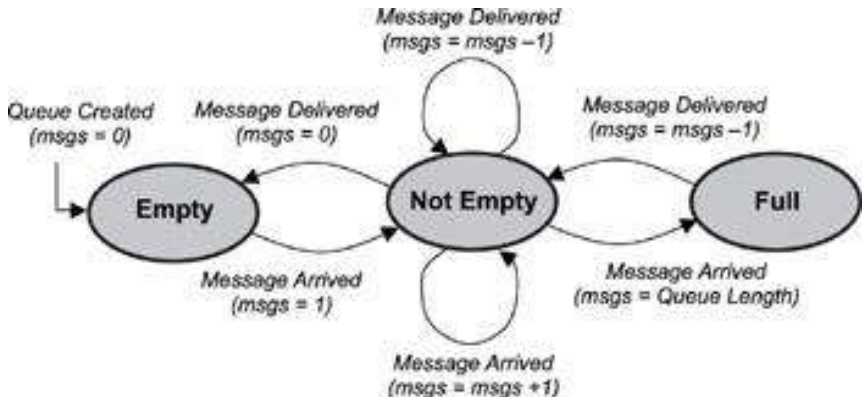
A mutual exclusion (mutex) semaphore is a special binary semaphore that supports ownership, recursive access, task deletion safety, and one or more protocols for avoiding problems inherent to mutual exclusion

23. Defining Message Queue

A message queue is a buffer-like object through which tasks and ISRs send and receive messages to communicate and synchronize with data. A message queue is like a pipeline. It temporarily holds messages from a sender until the intended receiver is ready to read them. This temporary buffering decouples a sending and receiving task; that is, it frees the tasks from having to send and receive messages simultaneously.

.

24. Draw the state diagram for a message queue.



25. List out the message queue content

Message queues can be used to send and receive a variety of data. Some examples include:

- a temperature value from a sensor,
- a bitmap to draw on a display,
- a text message to print to an LCD,
- a keyboard event, and
- a data packet to send over the network.

26. Define Message Queue Storage:

Different kernels store message queues in different locations in memory. One kernel might use a system pool, in which the messages of all queues are stored in one large shared area of memory. Another kernel might use separate memory areas, called private buffers, for each message queue.

27. What are the uses of message queues?

The following are typical ways to use message queues within an application:

- non-interlocked, one-way data communication,
- interlocked, one-way data communication,
- interlocked, two-way data communication, and
- Broadcast communication.

28. What is Preemptive Priority-Based Scheduling?

Most real-time kernels use preemptive priority-based scheduling by default. As shown in figure with this type of scheduling, the task that gets to run at any point is the task with the highest priority among all other tasks ready to run in the system.

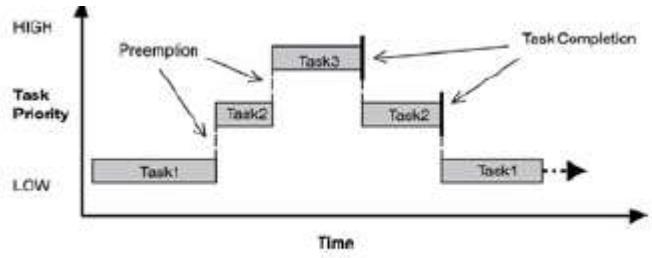


Figure: Preemptive priority-based scheduling.

29. What is Round-Robin Scheduling?

Round-robin scheduling provides each task an equal share of the CPU execution time. Pure round-robin scheduling cannot satisfy real-time system requirements because in real-time systems, tasks perform work of varying degrees of importance.

30. What is BLOCKED STATE?

A task can only move to the blocked state by making a blocking call, requesting that some blocking condition be met. A blocked task remains blocked until the blocking condition is met. (It probably ought to be called the *un*blocking condition, but blocking is the terminology in common use among real-time programmers.) Examples of how blocking conditions are met include the following:

- a semaphore token (described later) for which a task is waiting is released,

- a message, on which the task is waiting, arrives in a message queue, or
- a time delay imposed on the task expires.

31. What are the task operations that developers can perform?

- creating and deleting tasks,
- controlling task scheduling, and
- Obtaining task information.

32. What is meant by Single shared-resource-access synchronization?

One of the more common uses of semaphores is to provide for mutually exclusive access to a shared resource. A shared resource might be a memory location, a data structure, or an I/O device-essentially anything that might have to be shared between two or more concurrent threads of execution. A semaphore can be used to serialize access to a shared resource, as shown in Figure.

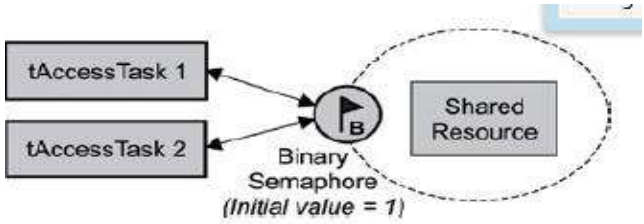


Figure: Single shared-resource-access synchronization

33. What is meant by Recursive shared- resource-access synchronization?

- Sometimes a developer might want a task to access a shared resource recursively. This situation might exist if tAccessTask calls Routine A that calls Routine B, and all three need access to the same shared resource, as shown in figure.

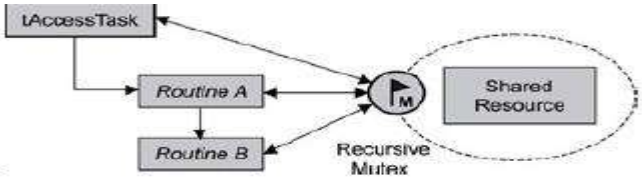


Figure: Recursive shared- resource-access synchronization.

34. Compare monolithic and micro kernel (May 2016)

Monolithic Kernel:

- Monolithic kernel is a single large process running entirely in a single address space.

- It is a single static binary file.
- All kernel services exist and execute in the kernel address space.
- The kernel can invoke functions directly.
- Examples of monolithic kernel based OSs: Unix, Linux.

Micro Kernel:

- In microkernels, the kernel is broken down into separate processes, known as servers.
- Some of the servers run in kernel space and some run in user-space.
- All servers are kept separate and run in different address spaces.
- Servers invoke "services" from each other by sending messages via IPC (Interprocess Communication). This separation has the advantage that if one server fails, other servers can still work efficiently. Examples of microkernel based OSs: Mac OS X and Windows NT.

35. Define Idle Process (May 2016)

System Idle Process contains one or more kernel threads which run when no other runnable thread can be scheduled on a CPU. Modern processors use idle time to save power.

36. Define RTOS. (May 2017)

A real-time operating system (RTOS) is an operating system (OS) intended to serve real-time applications that process data as it comes in, typically without buffer delays. Processing time requirements (including any OS delay) are measured in tenths of seconds or shorter increments of time.